Test Project

Module A Mini Speed Test Projects

Web Technologies

Thomas Seng Hin Mak
Leyla Santos BR
Valentin Adamescu UK
Seung-Iyul Ryu KR
Zhang Ling CN
Ryoju Ohata JP
Ebben Kapali NA
Manuel J. Schaffner CH
Kok Leong Tang SG

WSC2024_TP17_en

Contents

Introduction 3
Description of project and tasks 3
Tasks for Design Implementation 3
Tasks for Front-end Development 20
Tasks for Back-end Development 48
Instructions to the Competitor 57
Other 57

Introduction

In this mini speed test project, you are given multiple mini tasks. We have collected all the tasks from multiple clients. And for priority reason, we have selected some tasks for this round of development. Please only implement the required tasks.

Please follow the instruction to know which one to implement.

For each mini Test Project, there are three levels.

- Level 1: 0.5 points worth that is expected takes less than 15 minutes of work.
- Level 2: 1.0 points worth that is expected takes less than 30 minutes of work.
- Level 3: 1.5 points worth that is expected takes around 30 minutes of work.

Please put your files in /XX_module_a/ and with the task ID as folder name, e.g. /XX_module_a/A1/

The XX is the workstation number.

Description of project and tasks

Please create an index page to link to each mini speed Test Project.

The index page should contain thumbnail and title of each mini speed test project.

Please write at least one git commit for each mini speed test project. The git commit messages should be readable and easy to understand.

Tasks for Design Implementation

A1: Magnifying Dock (Level 1)

You are required to create a **magnifying dock** using HTML, CSS, and JavaScript. When the mouse hovers over the dock, the surrounding elements should be magnified. You can use a maximum distance of **340px** and a maximum scale of **1.3** as a reference point, but feel free to adjust these values to achieve a superior effect. Additionally, there should be a **glow effect** around the items that follows the cursor, enhancing the overall visual appeal.



© WorldSkills International

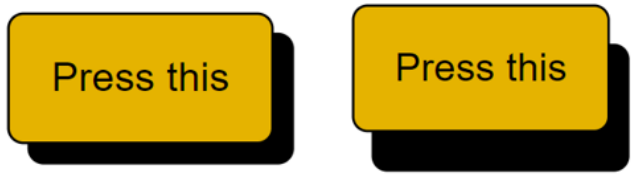
Requirement Notes

- A glow effect is added, which moves around the boxes with the cursor.
- The magnifying effect is at its maximum at the center of the box.
- Objects closer to the cursor are magnified, while those further away appear smaller as the mouse moves.
- The magnification gradually reduces from one box to another, with more than one box being magnified at a time.
- Mouse movement leads to a smooth transition in the magnifying effect.

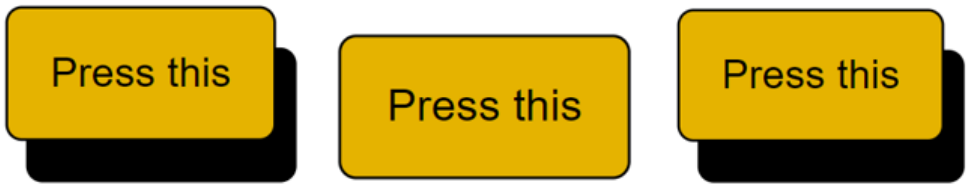
A2: Create a Neobrutalism animated button (Level 1)

Using just HTML and CSS, create a button that when hovering over it presents a floating effect as in example 01 and when clicking on the button, it presents an animation that represents the simulation of a suspended button being pressed as in example 02.

Example 01:



Example 02



The expected result must be the same as that presented in the " video-B1.mp4" file

Requirement Notes

- Button implemented using only HTML and CSS.
- Hover effect implemented.
- Active button effect implemented.
- When you release the click on the button, does it return to the main position.

A3: Animated link (Level 1)

Using only HTML and CSS, create an animated link that has two different animations for the two different behaviors.

- Hover:

Link para...

- Active:

Link para...

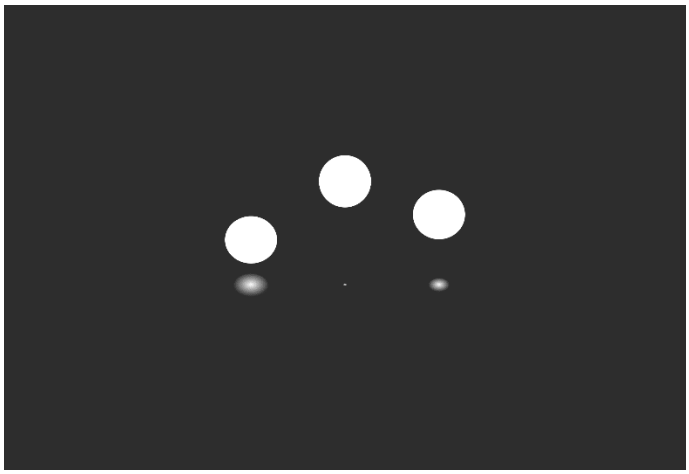
The expected result must be the same as that presented in the " example-B2.mp4 " file

Requirement Notes

- Implemented link hover animation
- Implemented active link animation
- Implemented the animations without using javascript

A4: jumping ball (Level 2)

Follow the provided video and replicate the motion/shadow effect of the three white balls. They must be animated in a coordinated way.



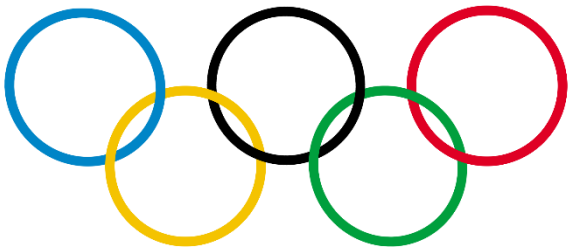
The expected result must be the same as that presented in the " example-B3.mp4 " file

Requirement Notes

- Recreated the balls alike the provided design
- Implemented the jump animation
- The animation have motion and shadow effects that follows the provided video
- The animation is happening in a coordinated way

A5: Olympic rings (Level 2)

Using only HTML and CSS, design the Olympic logo using the image provided as reference.



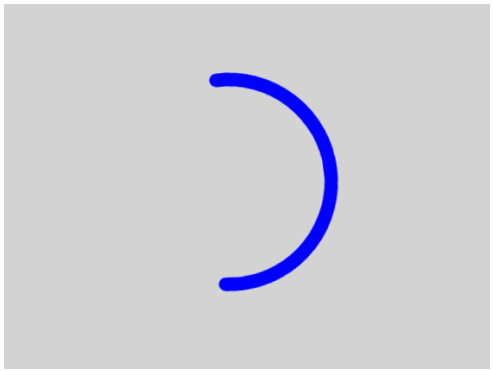
Requirement Notes

- The Olympics logo circles were created
- Circle overlay is similar to the provided media files

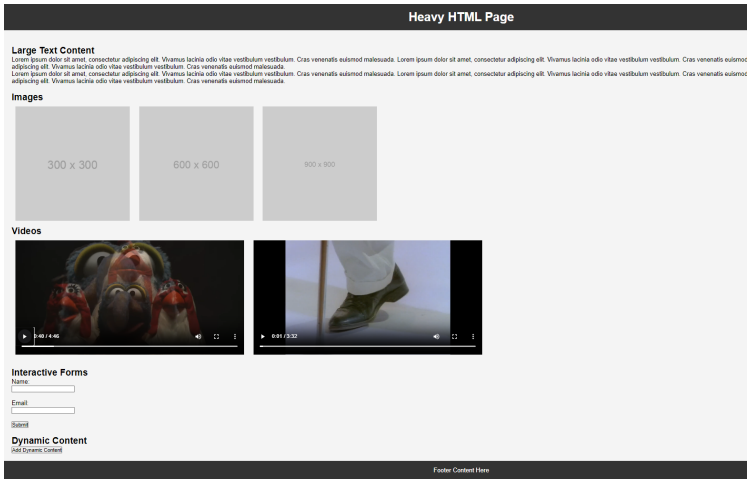
A6: Create a functional loading animation (Level 1)

Watch the video and recreate the spinner loading animation. The animation should only run while the provided HTML page is loading. The browser must be settled with a slow 3G connection and 6x slowdown CPU to test this.

Loading animation example:



Page displayed after animation:



Requirement Notes

- The loading animation have been created
- The loading animation follows the provided media files
- The loading animation is displayed only while the page is loading

A7: Create a custom cursor (Level 2)

According to the provided video and the image bellow, recreate the custom cursor with the same effect.



The expected result must be the same as that presented in the " example-C3.mp4 " file

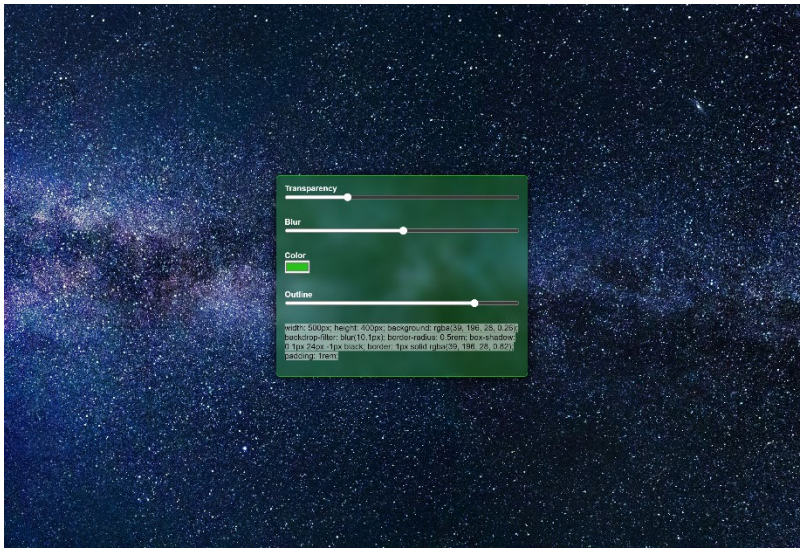
Requirement Notes

- The hover element has been created
- The hover element follows the shape showed provided media files
- The hover element replicates the effect provided in media files

A8: Create a glassmorphism card (Level 2)

From the provided HTML page, allow the user to change the properties of the glassmorphism card present in the file, using the input buttons presented in the example.

In addition to changing card properties, you must also display the modified CSS code at the bottom of the card.



A video for further reference will be provided (example-C4.mp4). The Competitor needs to add inputs to control

- Transparency (opacity)
- Blur
- Color
- Outline opacity

Requirement Notes

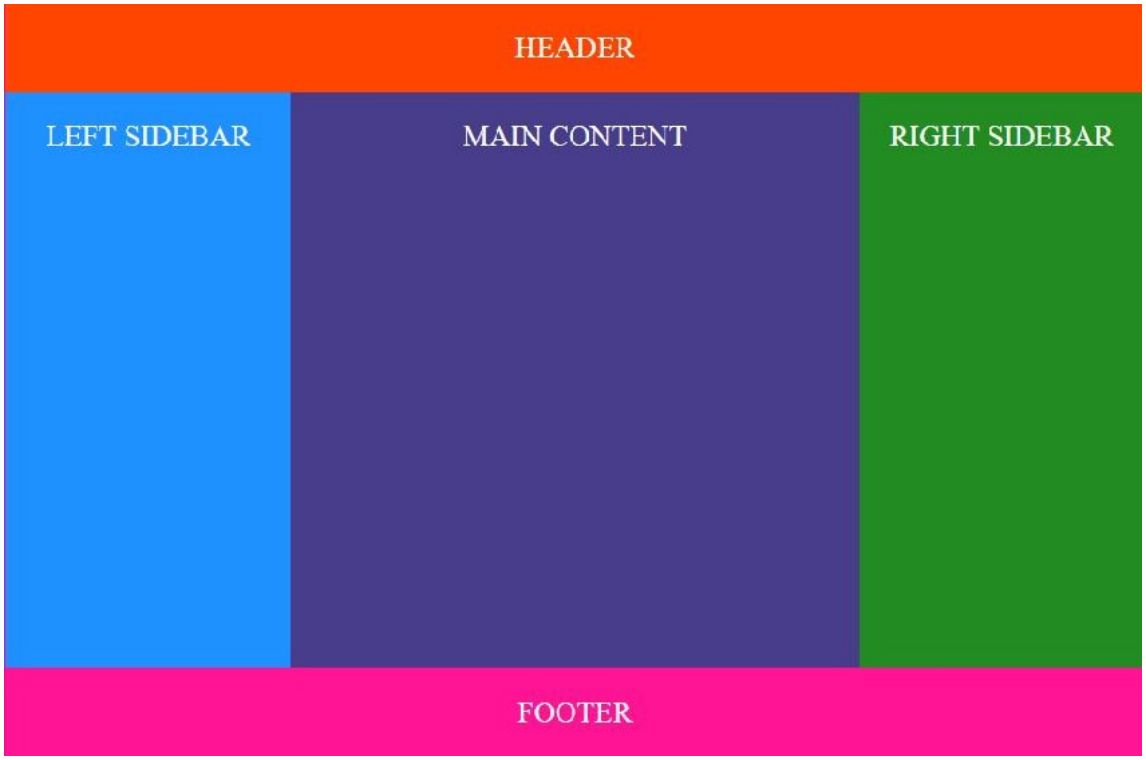
- The inputs have been implementd (Transparency (range), Blur (range), Color (color), Outline(range))
- The card changes according to the inputs value change (decrease 0.15 for each input not working)
- The provided CSS code follows the same implemented in the page

A9: Holygrail Layout (Level 1)

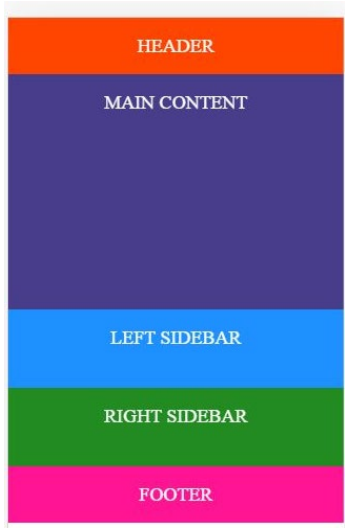
Holy Grail is a layout pattern that's very common on the web. It consists of a header, a main content area with fixed-width navigation on the left, content in the middle and a fixed-width sidebar on the right and then a footer.

Desktop:

© WorldSkills International

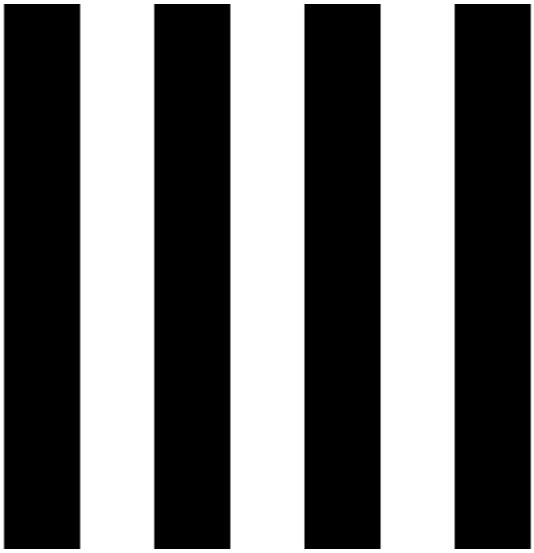


Mobile:



A10: Background Pattern (Level 1)

Using HTML, CSS creates a stripes background pattern with a 240px width and 240px height



A11: Colorful Button (Level 3)

Given the following HTML:

```
<button id="btn">Button</button>
```

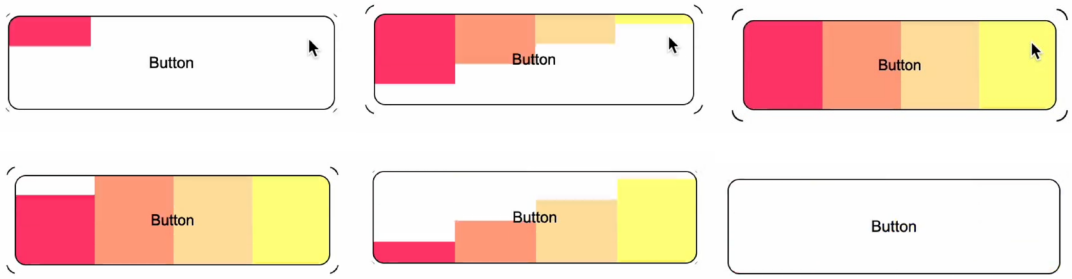
And the following CSS:

```
*{padding: 0;margin: 0;}
html,body{height: 100%;}
body{
    display: flex;
    justify-content: center;
    align-items: center;
}
:root {
    --bgc: #ff3366;
    --bgc2: #ff9977;
    --bgc3: #ffdd99;
    --bgc4: #ffff77;
}
#btn {
    position: relative;
    border-radius: 10px;
    overflow: hidden;
    width: 280px;
    height: 80px;
    margin: 30px;
```

```
    border: 1px solid;
    background-color: #fff;
}
```

Please implement the following hover effect for a button using CSS only, without modifying the HTML. When the mouse hovers over the button, four color bars should appear inside the button, moving in from top to bottom, starting from the left and going to the right. Simultaneously, four arcs should appear from the corresponding corners of the button, growing in size. When the mouse moves out of the element, the arcs should revert back to their original position, and the four color bars should move out from bottom to top, starting from the left and going to the right.

The effect video "colorful-button.mp4" is provided in the media files, and here are some sample images.



Requirement Notes

- When the mouse hovers over the button, four color bars appear inside the button, moving in from top to bottom, starting from the left and going to the right. Using CSS only, without modifying the HTML. The effect is same as the video
- When the mouse hovers over the button, four arcs should appear from the corresponding corners of the button, growing in size.
- When the mouse moves out of the button, the arcs should revert back to their original position.
- When the mouse moves out of the button, the four color bars should move out from bottom to top, starting from the left and going to the right.
- The four color bars do not cover the text on the button.

A12: Typing (Level 1)

Implement typing and text deletion animation based on the provided video "typing.mp4", and loop the animation.

You cannot use any JS or CSS libraries.

The main.js file provides an array of strings that you need to use. Do not modify the contents of this array. The contents are as follows:

The main.js file includes an array of strings that you'll need to use for this task. You can edit all the files provided, but please don't change the contents of the array in main.js. The array is defined as follows:

```
const texts = ['David Smith', 'Web Developer', 'Web Designer', 'Competitor of Worldskills'];
```



© WorldSkills International

I am Web D|

I am Web Developer|

I am We|

I am Web Designer|

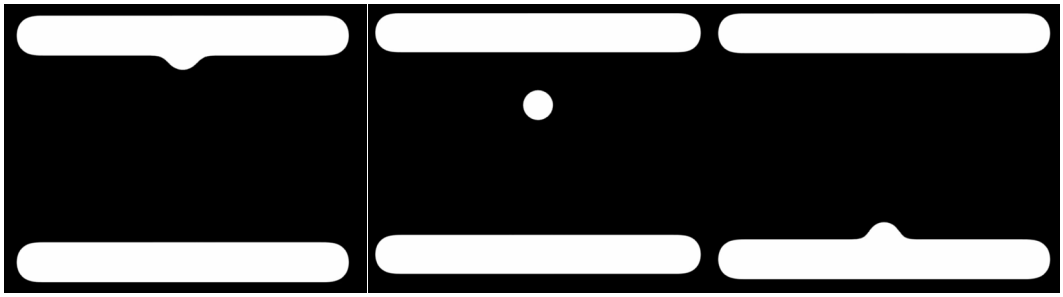
Requirement Notes

- Please make sure the typing animation and deletion animation runs correctly.

A13: Drip (Level 2)

No code is provided. Please implement a looping drip effect only using HTML and CSS, following the provided video.

These are some screenshots from the video.



Requirement Note

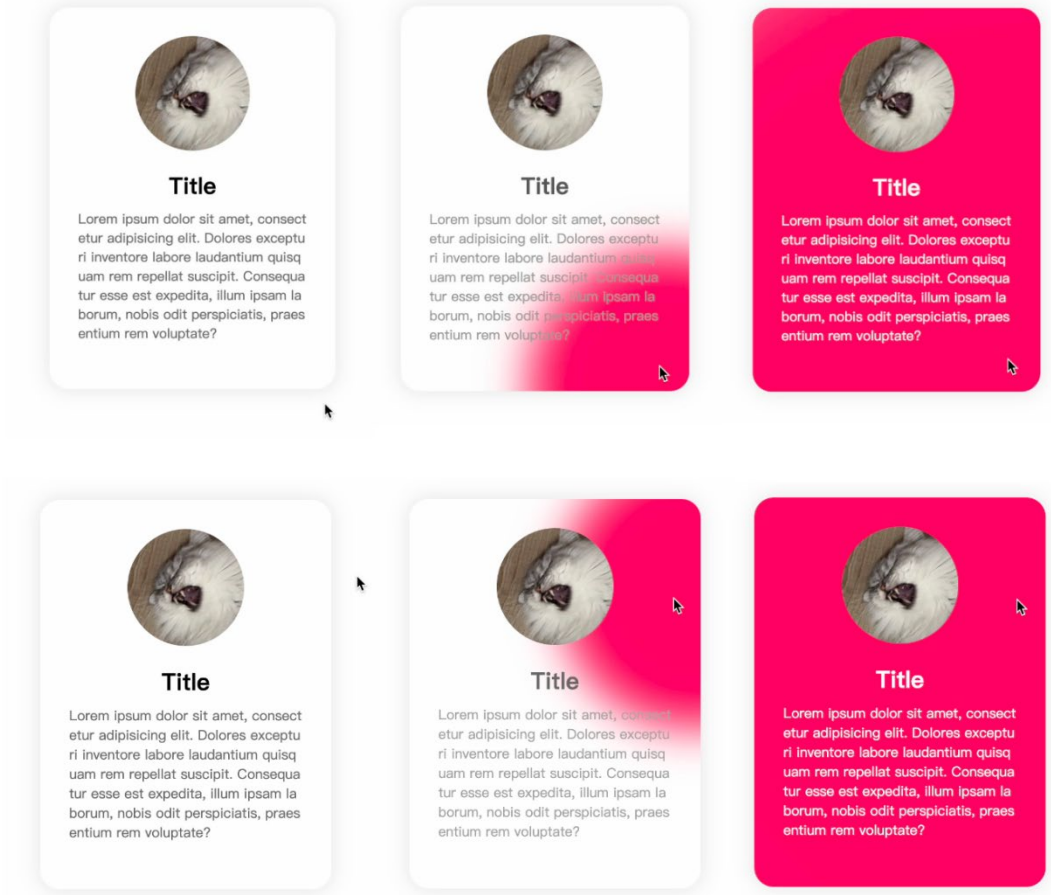
- Please refer to the reference video and create the same effect.

A14: Ripple Card (Level 2)

Implement a card displaying information, following the provided "ripple_card.mp4" video. When the user's mouse pointer enters the card, a ripple effect should appear from the direction of the mouse entry. The avatar.jpg and lorem text are provided.

You can only use HTML and CSS.

Here are some frames from the "ripple_card.mp4" video:



Requirement Notes

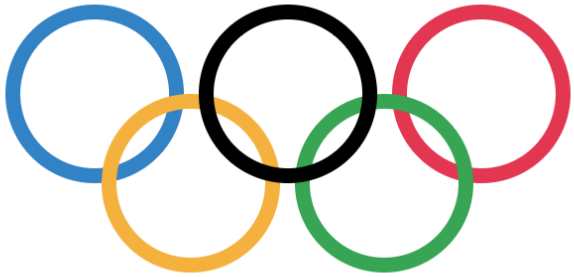
- The ripple effect appears from the direction the mouse enters the card.
- Only HTML and CSS were used.The effect is same as the video

A15: Olympics Games – (Level 1)

Using CSS only create the Olympic Games circles exactly as in provided sample.

The colours are: Blue: #0081C8; Black: #000; Red: #EE334E; Yellow: #FCB131; Green: #00A651;

The circles are exactly 100px and thickness is 10px.



Requirement Notes

- All five circles are present
- The correct colours are used (Blue: #0081C8; Black: #000; Red: #EE334E; Yellow: #FCB131; Green: #00A651;)
- The proportion is maintained
- The z-index is maintained (yellow over blue, black over yellow and green; green over red)

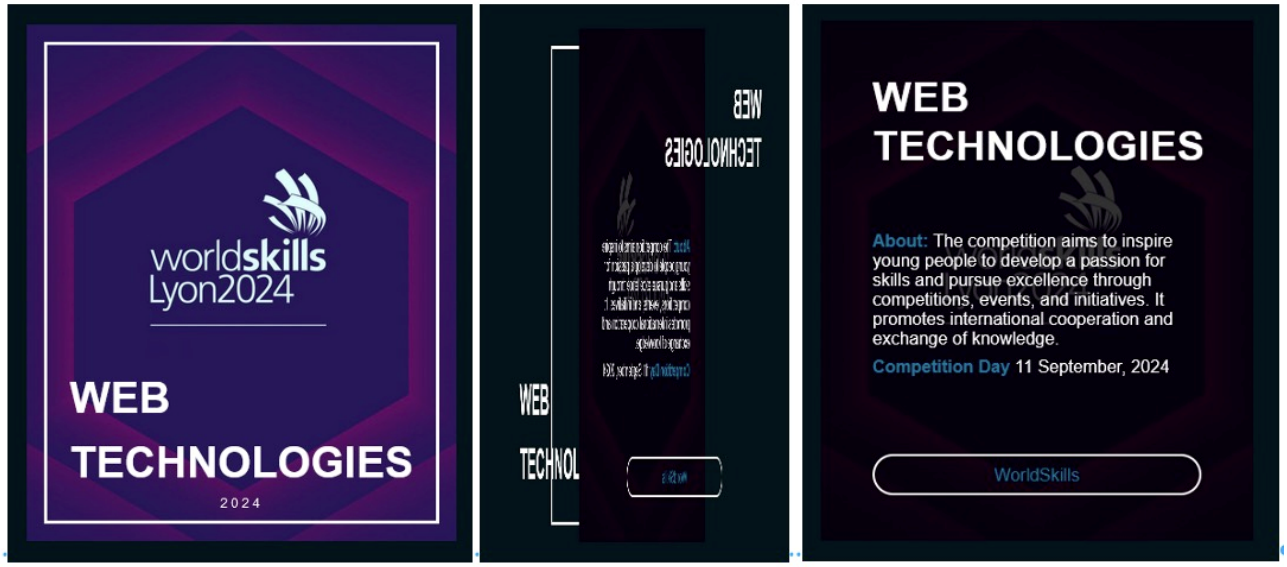
A16: 3D Flip card – (Level 3)

Using only CSS (JavaScript is not allowed) create a 3D flip card effect. You should use the provided fonts and colours and make the implementation as the provided samples.

Purple: #4a0d66; Blue #0e779f;

The card will flip from front to back when is hovered, creating a 3D effect text. The front border and the text 'Web Technologies' must show as a "floating element" when is making the rotation; The rear side will have the text and the button as a 3D element.

The required effect is presented in the media_sample/3d_flip_card_sample.mp4.



Requirement Notes

- The front of the card is similar with the provided sample
- The back of the card is similar with the provided sample
- The front of the card has the 3D elements of border and text
- The back of the card has the 3D elements of title and button

A17: Simple Cube Rotation – (Level 2)

You will need to implement a simple cube rotation with the following conditions:

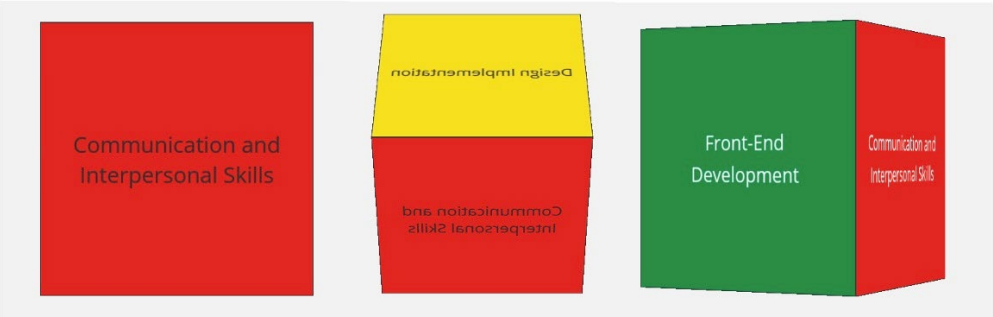
CSS only JavaScript is not allowed.

Colours red: #e1251b; green: #008c45; yellow: #f7e017; The colours must be in opposite directions over the all 6 cube faces.

- Red for front/back (Communication and Interpersonal skills + Work organisation and management)
- Yellow for left/right (Design implementation + Speed)
- Green for top/bottom (Back-end Development + Front-end Development)

The rotation should start with "Work organisation and management" and will be clock-wise/top-bottom for each of the faces. All 6 faces must be shown.

The required effect is presented in the media_sample/simple_cube_rotation_sample.mp4.



Requirement Notes

- The cube is rotating using CSS only no JS
- All 6 faces are shown at least once
- The cube is rotating in the requested sequence 1-clock-wise/1 Down
- The faces are matching as requested colours

A18: Carousel - Key control (Level 2)

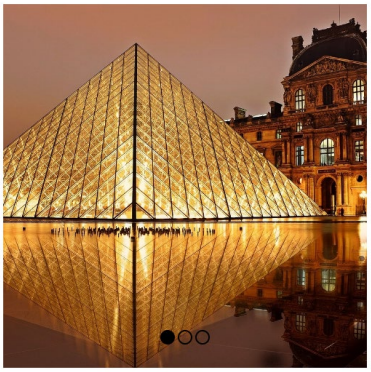
Create a carousel using the given images.

There are buttons at the bottom center to control the slides, and when you press the buttons, it moves to the corresponding order.

Click on the carousel, pressing the arrow keys (left, right) slides it left or right. (It doesn't move left in the first image and doesn't move right in the third image)

Buttons that allow you to manipulate slides change their background color depending on the current slide situation.

(You can use html and css, but not javascript)



Requirement Notes

- The carousel area is implemented and uses the provided images.
- There are clickable buttons at the bottom center of the carousel area that allow you to manipulate the slides, and the style changes depending on the slide situation.
- When you click the button, it will move in that order.
    - Click on the carousel area and press the arrow keys to move left and right. (It doesn't move left in the first image and doesn't move right in the third image)

A19: Flag Animation (Level 3)

Please implement the following waving French flag.



The provided html and css can be modified, but javascript cannot be used

Refer to the demo.mp4

Requirement Notes

- The French flag provided was used.
- All parts of the flag are animated seamlessly and naturally.
- The animation effect continues indefinitely.

A20: CSS Line Chart (Level 2)

Please implement a line chart using only HTML and CSS.

At first, all values are 0 (the lines and points are at the bottom), but when you hover over the chart area, the lines and points rise from the bottom.

The four items have values from left to right: 65%, 25%, 85%, 40%.

You can use html and css, but not javascript

Please refer to resource/demo.mp4

Requirement Notes

- There is a chart area, and lines and points are implemented within it.
- Initially, the lines and dots are located at the bottom.
- When you hover your mouse over the chart area, lines and strokes will rise from the bottom as shown in the provided video.
- When you hover your mouse over the chart area, All items will rise to the specified height. 0
- The point is located exactly where the line bends.

A21: 3D Button (Level 1)

Using the provided video as a reference, create a button with hover animation. JavaScript cannot be used.

Requirement Notes

- Animation matches the sample video
- Text changes on hover

A22: CSS Bar Chart (Level 1)

Please implement a bar chart using only HTML and CSS.

At first, the screen will be blank, but when you hover the mouse over the chart area, the bars will rise sequentially from the bottom, starting from the left bar.

The values and colors of each data point are not important.

You can use html and css, but not javascript

Please refer to resource/demo.mp4

Requirement Notes

- The bar graph area is displayed and the initial screen is blank.
- When you place the mouse over the chart area, the bars move up in order from the bottom.
- When you place the mouse over the chart area, the bar goes up sequentially starting from the left.

A23: Scroll Animation (Level 2)

Write animation.css so that the following animations work simultaneously with page scrolling. (Refer to demo.mp4)

You cannot modify any files other than animation.css.

Section 1

The background narrows and the text disappears.

Section 2

When the Section 2 area fully fills the screen, it scrolls horizontally to the right.

Section 3

After all content in Section 2 is scrolled, it scrolls normally, revealing the text.

Requirement Notes

- Implement the functions of Section 1 as shown in the provided video.
- Implement the functions of Section 2 as shown in the provided video.
- Implement the functions of Section 3 as shown in the provided video.
- The function was implemented by only modifying the provided animation.css.

A24: Loading animation (Level 2)

Please create a continuously repeating loading animation by referring to the given images and videos.



You can use html and css, but not javascript.

Please refer to demo.mp4

Requirement Notes

- There are two X-axis and Y-axis bars, as shown in the image provided.
- Pay attention to the z-index and There is an animation effect similar to the provided video.
- The animation continues indefinitely.

A25: CSS Pie Chart (Level 1)

Please implement a pie chart using only HTML and CSS.

At first, it will be a blank screen, but when you hover over the chart area, the pie chart will appear in a clockwise direction, and when the mouse leaves the chart area, the chart will disappear in a counterclockwise direction.

The three items have values of 45%, 25%, 30% clockwise.

You can use html and css, but not javascript.

Please refer to resource/demo.mp4

Requirement Notes

- The pie graph area is displayed and the initial screen is blank. (Except immediately after first loading)
- When you place the mouse over the chart area, the chart appears clockwise, and when you move the mouse away, the chart disappears counterclockwise.
- The appearance of the chart is the same as the provided video.
- The values of the chart items are 45%, 25%, 30%.

A26: Dashboard Layout (Level 2)

Please modify the provided file to create the layout as shown in the image below.

You can only modify the `your-work.css` file and cannot change any other files.

There should be no scrolls except for the list scroll inside the section, and the list content should not overflow the section.



© WorldSkills International

Requirement Notes

- There is no other scrolling except for the list scroll inside the section.
- The list content does not exceed the section and scrolls.
- The functionality was implemented by only modifying the provided your-work.css file.

A27: Accessibility (Level 2)

A web page is provided. The web page contains five types of accessibility issues.

Resolve all accessibility errors and place the file in the `resolve` directory.

Requirement Notes

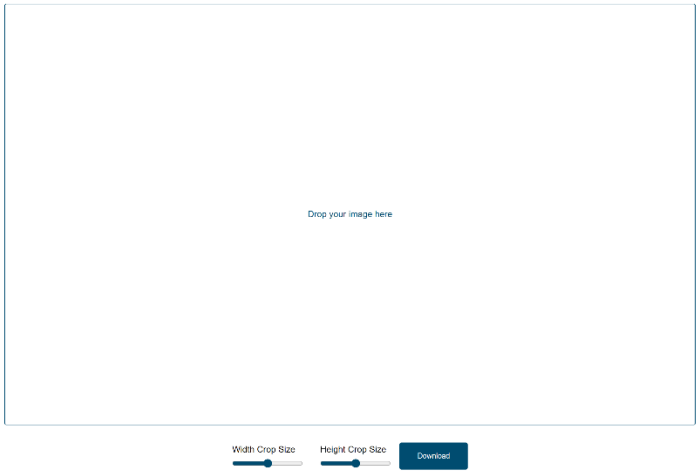
- Fixed the minimum and maximum sizes of the viewport.
- Used the button tag.
- Fixed accessibility issues with image tags.
- Modify visibility and hover style of A tag
- Contrast accessibility issues fixed.
- Placed the files that resolved the accessibility issues in the resolve folder.

Tasks for Front-end Development

B1: Create image cropper (Level 3)

Given the provided files, recreate the image cropper given the following rules:

- The images must be uploaded through drag and drop and the canvas element size must not be changed according to the uploaded image size
- The crop canvas must show the image with a darken filter as background and a crop window (image without any filters)
- The crop window can be moved with the mouse cursor and must not exceed the darken image background. Also, the crop window size (width/height percentage of the image) can be changed through provided range inputs
- The download button has to be functional and the downloaded image must follow the original size of the uploaded image



The expected result must be the same as that presented in the "example-C5.mp4" file

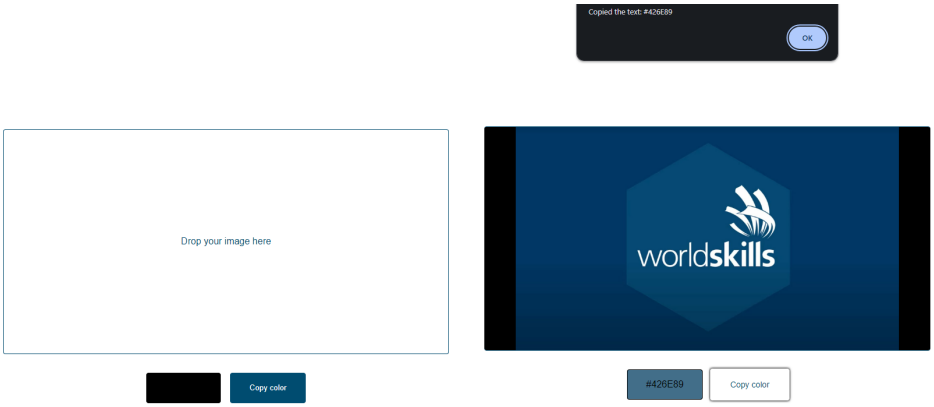
Requirement Notes

- The images must be uploaded through drag and drop and the canvas element size must not be changed according to the uploaded image size
- The crop canvas must show the image with a darken filter as background and a crop window (image without any filters)
- The crop window can be moved with the mouse cursor and must not exceed the darken image background. Also, the crop window size (width/height percentage of the image) can be changed through provided range inputs
- The download button has to be functional and the downloaded image must follow the original size of the uploaded image

B2: Create image color picker (Level 3)

Given the provided files, recreate the image color picker given the following rules:

- The images must be uploaded through drag and drop and the canvas element size must not be changed according to the uploaded image size
- The canvas must have a cursor that shows the current color (pixel) on hover
- When the user clicks on the canvas, the current color is displayed in the bottom output area, along with its hex code
- The copy color button must copy the color to the clipboard and warn the user



The expected result must be the same as that presented in the " example-C6.mp4 " file

Requirement Notes

- The images must be uploaded through drag and drop and the canvas element size must not be changed according to the uploaded image size
- The canvas must have a cursor that shows the current color (pixel) on hover
- When the user clicks on the canvas, the current color is displayed in the bottom output area, along with its hex code
- The copy color button must copy the color to the clipboard

B3: Fixed-Column Table (Level 2)

Create a table of 100% width, 15 columns with fixed width of 100px. When it overflows, only the first, second and last column remains fixed and will not move when scroll as shown. Use only JS, HTML and CSS for this question. You can use any text for thead and tbody.

Id	Merchant Name	Platform Fee	Platform Dispute Fee	Deductions	Payout	Error #	Status	Actions
10	UNKNOWN	USD 0.79	USD 0.00	USD 0.00	USD 12.47	1	Error	
11	Developer1	USD 2.38	USD 0.00	USD 0.00	USD 37.41	1	Await	Void
12	Developer2	- USD 0.67	- USD 0.67	- USD 38.31	USD 2.90	0	Await	Validate Void
7	UNKNOWN	USD 0.79	USD 0.00	USD 0.00	USD 12.47	1	Error	
8	Developer1	USD 2.38	USD 0.00	USD 0.00	USD 37.41	1	Await	Validate Void
9	Developer2	- USD 0.67	- USD 0.67	- USD 38.31	USD 2.90	0	Await	Validate Void
4	UNKNOWN	USD 0.79	USD 0.00	USD 0.00	USD 12.47	1	Error	

B4: 3D Gallery (Level 2)

Given the HTML:

```html
<div class="container">
    <div class="img-box">
        <img src="imgs/1.jpeg" alt="">
    </div>
    <div class="img-box">
        <img src="imgs/2.jpeg" alt="">
    </div>
    <div class="img-box">
        <img src="imgs/3.jpeg" alt="">
    </div>
    <div class="img-box">
        <img src="imgs/4.jpeg" alt="">
    </div>
    <div class="img-box">
        <img src="imgs/5.jpeg" alt="">
    </div>
</div>
```

Given the CSS:

```css
*{
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

html,body{
    height: 100%;
}
body{
    display: flex;
    justify-content: center;
    align-items: center;
}

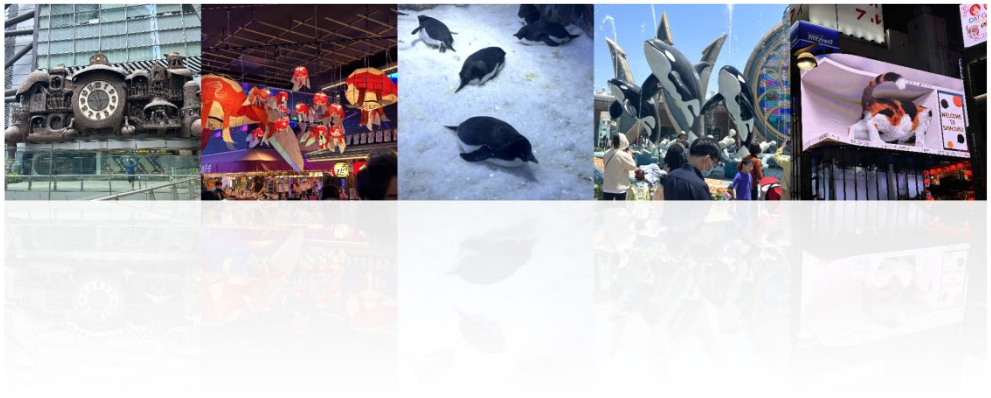
.container{
    display: flex;
}

.img-box{
    width: 200px;
    height: 200px;
    position: relative;
    transition: 0.5s;
}

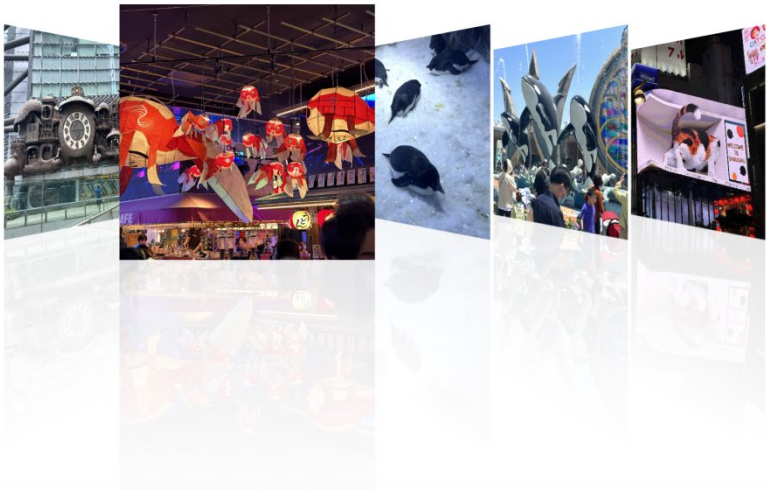
.img-box img{
    width: 100%;
    height: 100%;
    object-fit: cover;
    transition: 0.5s;
}
```

Develop a gallery using CSS only, where each image has a mirrored shadow image at its bottom. The gallery's interactive effects should follow the "3d-gallery.gif" file. When the mouse hovers over any image in the gallery, that image should enlarge for the user, while the other images in the gallery should tilt towards the hovered image.

You can only edit the style.css file. To make things as fast as possible, you don't need to worry about compatibility. It just needs to work correctly in one of the following browsers: Chrome, Firefox, or Edge.



not hovered



hovered

Requirement Notes

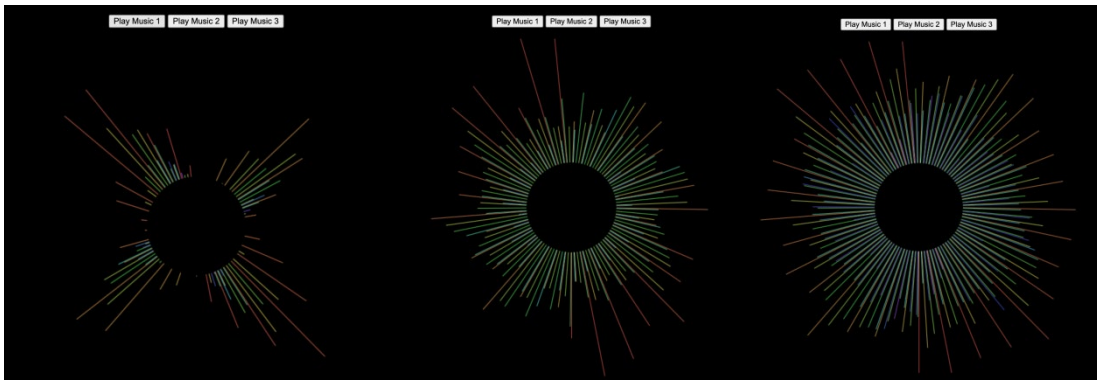
- Each image has a mirrored shadow image at its bottom. The HTML structure was not modified; only CSS code was written.
- The mouse hovers over any image in the gallery, that image should enlarge for the user.
  When an image is hovered over, other images in the gallery should tilt towards the hovered image.

B5: Music Waves (Level 2)

No code is provided. Please implement an animation effect that fluctuates along with the melody of the currently playing music. The fluctuation should be composed of many lines, surrounding a black circle. Each fluctuate line should move up and down from the position of the black circle, based on the music melody. Three MP3 files are provided, and you can create three buttons to play these music. Note that only one music can be played at the same time.

The three MP3 files are named: music-1.mp3, music-2.mp3 and music-3.mp3.

Here are some sample images.



Requirement Notes

- Only one music can be played at the same time.
- Each music file can be played by its corresponding button.
- The fluctuation should be composed of many lines, surrounding a black circle.
- Each fluctuate line should move up and down from the position of the black circle, based on the music melody.

B6: Direction Modal(Level 2)

Given the HTML:

```
<button class="open-modal position-absolute top-left">Show Modal</button>
<button class="open-modal position-absolute top-right">Show Modal</button>
<button class="open-modal position-absolute bottom-left">Show Modal</button>
<button class="open-modal position-absolute bottom-right">Show Modal</button>

<div class="modal">
    <div class="modal-content">
        <div class="modal-header">
            <div class="modal-title">Title</div>
        </div>
        <div class="modal-body">
            Lorem ipsum dolor sit amet, consectetur adipisicing elit.
            Ad aut corporis delectus doloribus eius eum exercitationem explicabo
            inventore iure laudantium natus perspiciatis praesentium quae quasi
            quo quod reprehenderit sint, sunt.
        </div>
        <div class="modal-footer">
            <button class="modal-btn">Close</button>
        </div>
    </div>
</div>
```

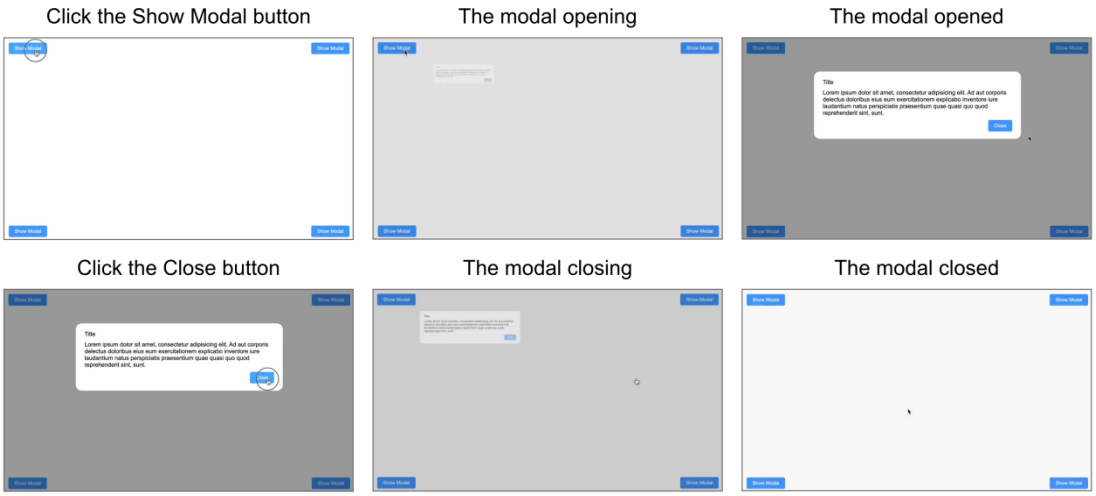
Develop a modal functionality that follows the effects shown in the "direction-modal.mp4" video, when the user clicks the “Show Modal” button to open the modal, the modal will fly into the interface from the button's position, with a fade-in and scale-in animation effect. When the user clicks on the modal's black transparent

background or the “Close” button, the modal should fly back to the button that opened it, with a fade-out and scale-out animation.

You cannot use any JS or CSS libraries.

You can only edit the "competitor.css" file and the "main.js" file.

The modal's interactive effects are as follows:



Requirement Notes

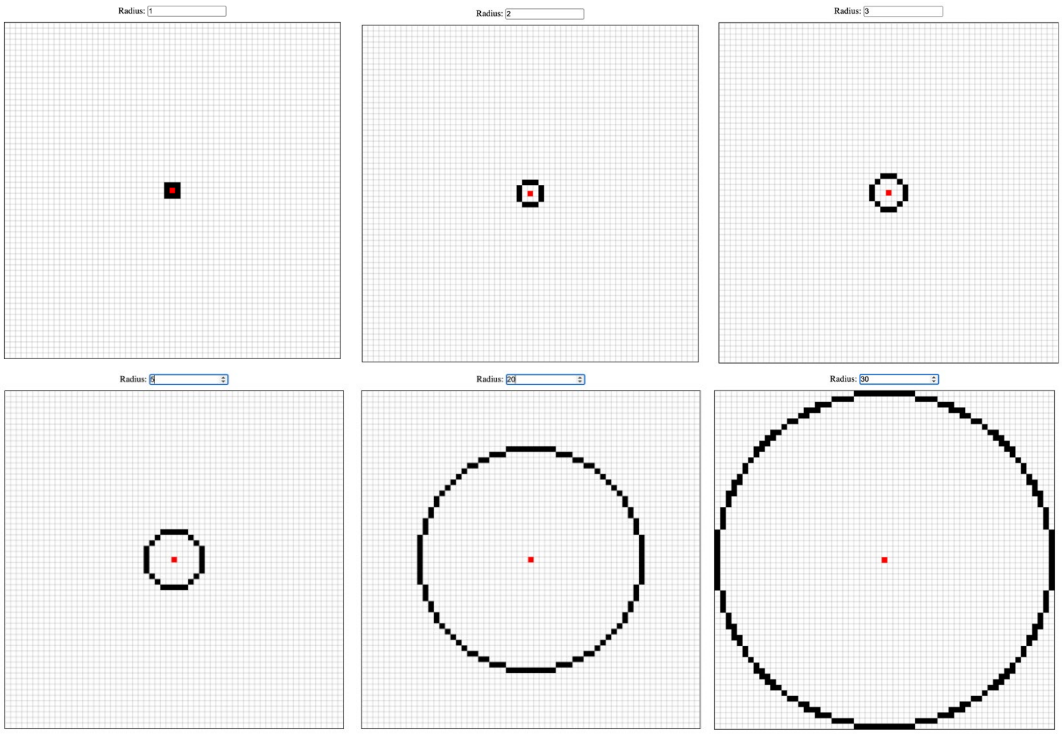
- The modal entry effect is correct: When the user clicks the “Show Modal” button to open the modal, the modal will fly into the interface from the button's position, with a fade-in and scale-in animation effect. Only edit the "competitor.css" file and the "main.js" file. The effect is same as the video.
- The modal leave effect is correct: When the user clicks on the modal's black transparent background or the “Close” button, the modal should fly back to the button that opened it, with a fade-out and scale-out animation.

B7: Pixel Circle Generator (Level 2)

No code and media files are provided. Please implement a pixel-style circle generator. In an input box, users can enter the radius of a circle, and the generator will then draw the corresponding circle on a canvas. The canvas should be composed of a 61x61 grid of squares, with each square measuring 10 pixels.

Here are some sample images.

© WorldSkills International



Requirement Notes

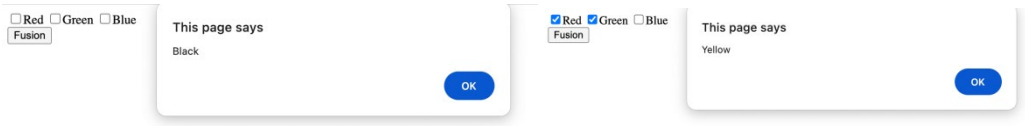
- The canvas should be composed of a 61x61 grid of squares.
- The circle shape can be correctly generated using the radius input field.
- When the radius is 30, the circle fills the entire canvas.

B8: Fusion (Level 1)

Develop a color fusion, where there are three checkboxes on the page corresponding to "Red", "Green", and "Blue" colors, and a "Fusion" button. Upon clicking the "Fusion" button, an alert will pop up based on the selected checkboxes, according to the following conditions:

Checked Color(s)	Alert Message
Red, Green and Blue	White
Red and Green	Yellow
Red and Blue	Purple
Green and Blue	Cyan
Red	Red
Green	Green
Blue	Blue
No checked	Black

No media files are provided. You must write the code yourself, without using any JavaScript libraries.



Requirement Notes

- All conditions and results described in the test project are displayed

B9: Image Compare(Level 2)

Given the HTML:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Image Compare</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div>
        <input type="text">
        <img src="">
        <img src="">
    </div>
</body>
</html>
```

You need to use the provided index.html, style.css (empty file), two images (before.jpg and after.jpg) and a splitter.svg file to implement the Image Compare functionality. The Image Compare container has a splitter element. This element displays before.jpg on the left and after.jpg on the right. You can adjust the display ratio of the two images by clicking on the container or dragging the splitter element. As you adjust the ratio, one image will be displayed more prominently, while the other will be displayed less.

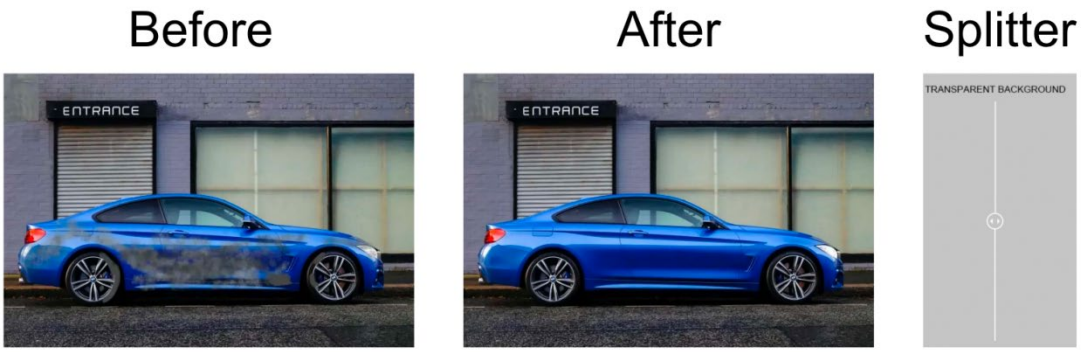
You cannot add or remove any HTML elements (except pseudo-elements). You can modify the attributes of existing HTML elements (including event listener attributes).

A sample video, "compare.mp4", is provided for reference. Below are some frames from the video:



The media files sample:

© WorldSkills International



Requirement Notes

- The splitter left is before.jpg, the splitter right is after.jpg. No HTML elements were added or removed.
- The splitter can be moved by dragging with the mouse or clicking within the container.
- The image's effect is same as the compare.mp4 video.

B10: Countdown clock (Level 3)

Create a webpage that displays a countdown clock for the voting period in a general assembly with a circular progress bar as shown in the short movie clip. The duration of the countdown should be hardcoded in one place only and should be 20sec. Color should change from green to orange at 10sec and to red at 5sec left. No error-handling required. Use plain HTML, CSS and JS.

Requirement Notes

- Countdown seconds number display working in center of circle
- Countdown circular progress bar working
- Countdown progress bar smooth (not jumping)
- Color changes green > orange 10sec > red 5sec
- Countdown duration changeable in one place is working
- Using plan HTML, CSS, JS

B11: Dynamic content display (Level 2)

Create a webpage that displays content from a json-file (example provided) for showing voting results during a general assembly.
Results contain a title/topic and the results for each vote: pro, con, abs. For each, the percentage and the total of votes needs to be calculated and displayed according to the image provided. The majority needs to be displayed in bold and green. No error-handling required. Needs to work loading from file and via webserver URL in Chrome.

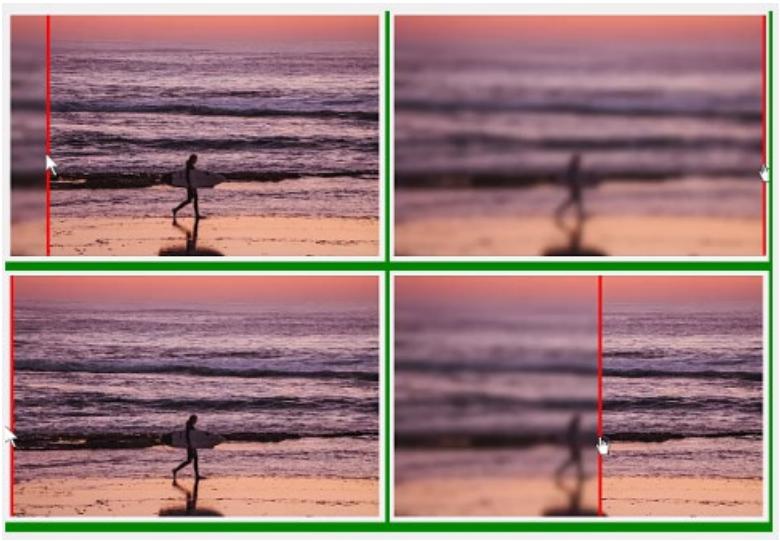
Requirement Notes

- Tabular display of (dynamic) values for title/topic pro, con, abs from JSON
- Percentages and total votes calculated from JSON data and shown
- Majority highlighted in bold and green
- Working loaded from local file and via webserver

B12: Image blur linear slider (Level 2)

Using only JavaScript create a linear slider that will blur an imagine progressively left and right. The provided CSS should not be modified. To drag the slider, you will need to click& drag on the red line left and right apply/removing the blur effect progressively. The slider should start as default from 50% of the imagine ad page load.

The required effect is presented in the media_sample/blur_slider_sample.mp4.



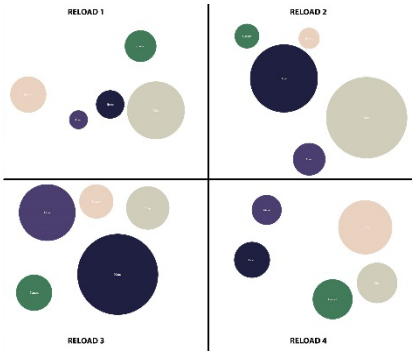
Requirement Notes

- The slider can be dragged
- The blur effect changes progressively from left to right (and right to left) as the slider is moved
- The slider interaction is smooth and responsive with minimal delay or lag (as provided media)
- Usage only of JavaScript and the provided CSS was not altered or additional CSS created

B13: Random Circles (Level 2)

Using JavaScript, create a page, that contains 5 circles. Each circle contains one of the following texts: *Home*, *About*, *Contact*, *Map, Privacy*, the texts are cantered both horizontally, and vertically. Position of the circles are random on the page, but no overlap allowed. The whole area of the circle must be visible and if the page is resized the new circles must fit in the new window size. The size of the circles is random between 100-500px on each page reload/refresh.

The required effect is presented in the media_sample/random_circles_sample.mp4.



© WorldSkills International

Requirement Notes

- All five circles are displayed on page load
- The circles have different dimensions
- The circles are not overlapping after page reload (reload page multiple times for testing)
- The circles are responsive – resize the window and the new circles should fit in the new window size

B14: Gradient selector (Level 3)

Create the JavaScript code for the following task:

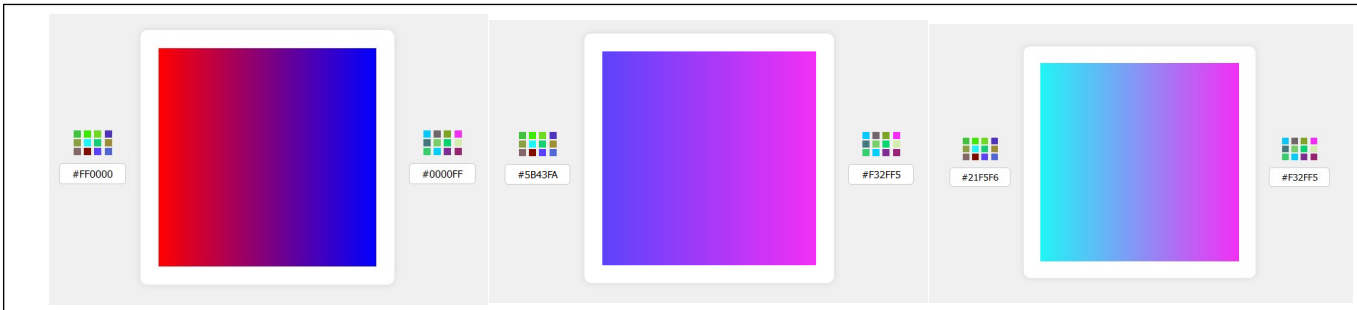
In the centre of the screen, there is a 300px square. By default, it will have a red to blue linear gradient background. On the left and right of the square, there are input fields where you can change the starting and ending colours of the linear gradient by entering the HEX value of the colour.

Above each input, there are 12 randomized colour buttons (10px by 10px) on each side. If you click one of the randomized colours, its colour is inserted into the input. Whenever any input is changed and contains a valid HEX value, the corresponding side's linear gradient colour on the square changes.

If you manually input any valid HEX value, the colour should change to the given value. You are not allowed to modify the HTML or CSS files, only JavaScript.

The required effect is presented in the provided media_sample/gradient_selector_sample.mp4.

Requirement Notes



- There are 12 randomized colour buttons on each side, and clicking a button inserts its colour into the respective input field.
- The 12 colours buttons on each side are visually organized in a 4x3 grid above the input fields.
- Input fields correctly update the corresponding side's gradient colour when a valid HEX value is entered .
- The square's gradient updates dynamically to reflect the colours entered or selected from the colour buttons, without requiring a page refresh.

B15: String reverser (Level 1) (JS Version)

Using JavaScript, create the following task: take a string input (e.g., "worldskills") from a user via an HTML form, reverse the string, convert it to uppercase, and display the result on the same page without reloading.



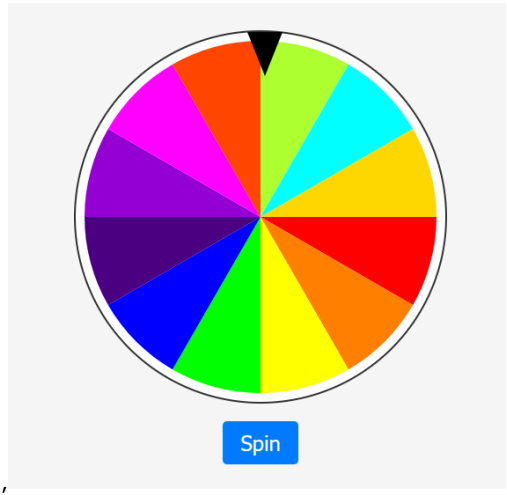
Requirement Notes

- The form correctly submits the input string.
- The input word is converted to upper case
- The inputted word is reversed
- The processed string is updated dynamically without requiring a page refresh

B16: Fortune wheel (Level 3)

Using CSS/JavaScript create a fortune wheel that will have 12 multicolour slices and will spin clock-wise when the **Spin** button is clicked and will stop after approximatively 5-6 seconds.

The required effect is presented in the provided media_sample/fortune_wheel.mp4



,

Requirement Notes

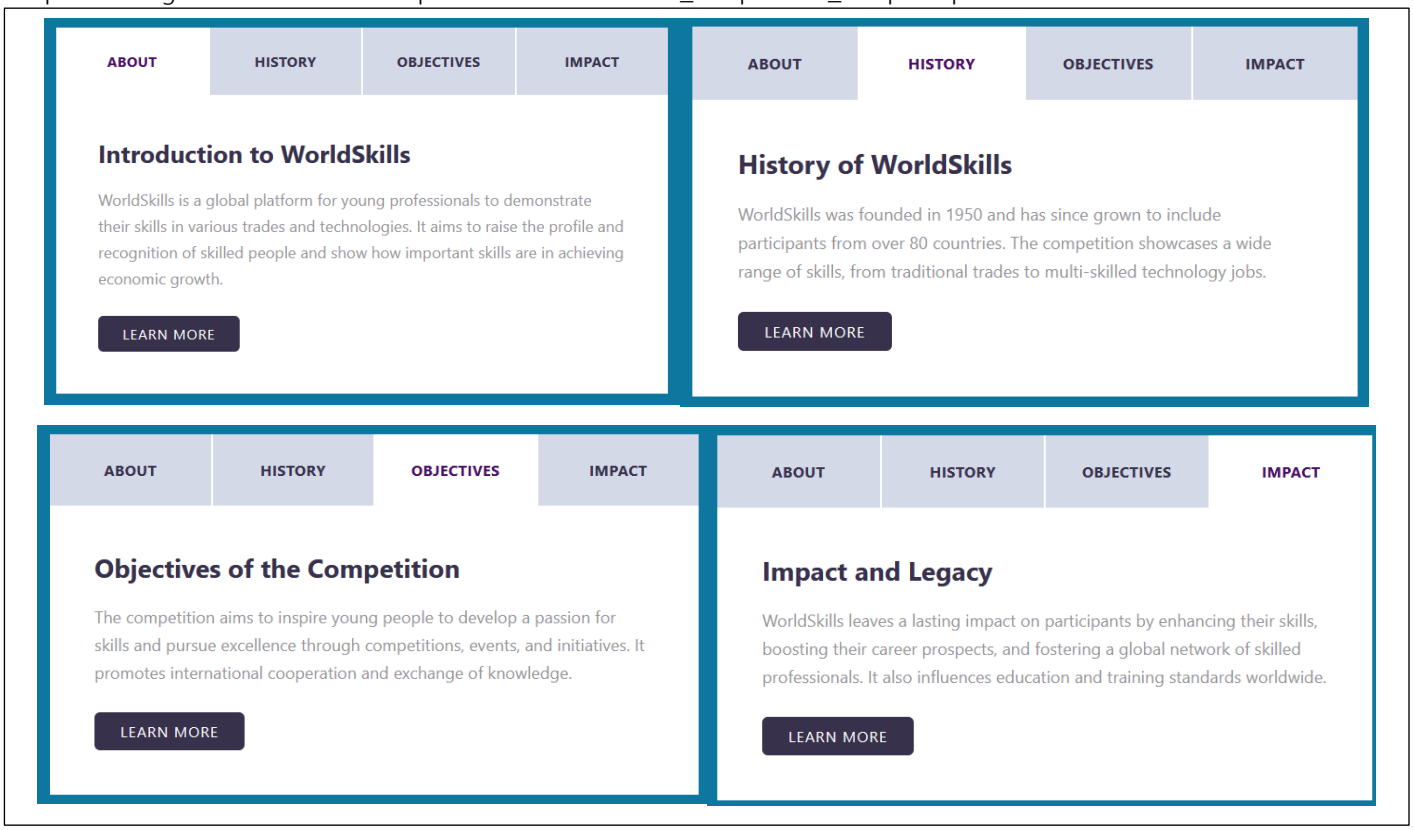
- There are 12 slices on the wheel
- The slices are behind the black pin
- The wheel spins when spin button is clicked as in provided
- The spin is about 5-6 seconds

B17: Tabs – (Level 2)

Your task is to create a tab menu with the provided text that will look exactly as the provided samples. No Bootstrap or Tailwind is allowed and you can use JavaScript if you wish so but no frameworks (vanilla).

Used colours are background: #0E779F; primary: #4A0D66; text-light: #969499; text-dark: #37314c; neutral: #d4d9e8;

The navigation between tabs should be smooth and the "Learn More" button have a hoover effect. The required navigation and effects are presented in the media_sample/tabs_sample.mp4.



Requirement Notes

- All four tabs are present with correct labels and colours (about, history, objectives, impact) (Treat as an error for any missing tab or other discrepancies)
- All 4 tabs are clickable and works as intended
- All text is correct displayed in their respective tabs (title and content)
- The 'Learn more' button have a hoover effect

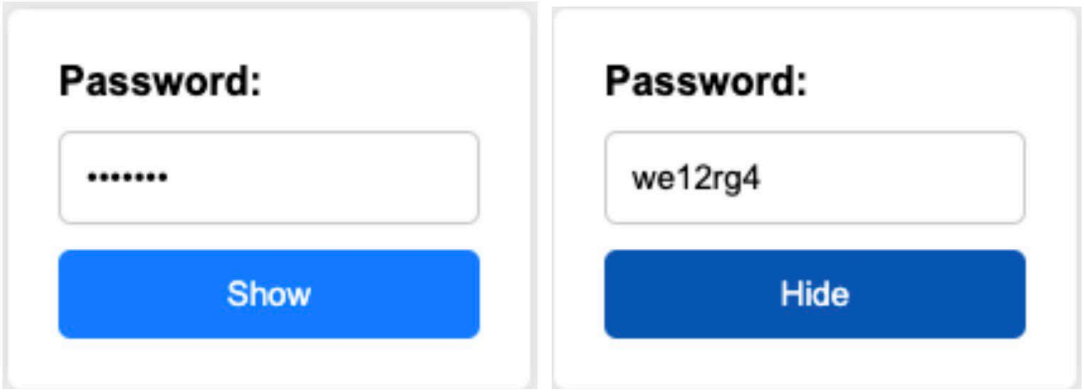
B18: Rotating Gallery (Level 2)

Without touching the CSS file, work on the HTML page and create a JavaScript file that adds functionality to rotate images in an image container when the "Next button or Previous button" is clicked.

Please refer to the example video in the media files.

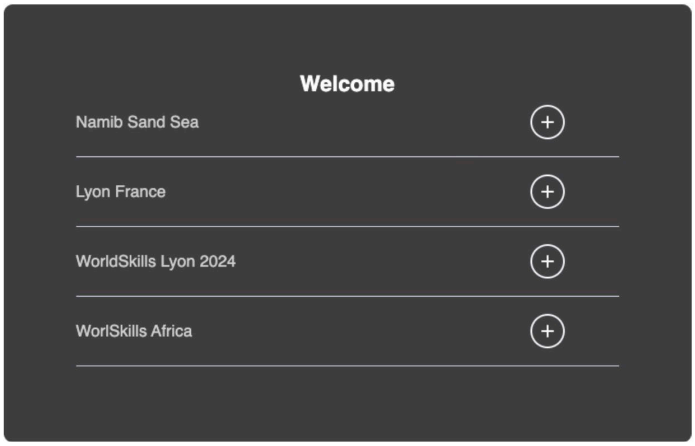
B19: Show and hide password (Level 2)

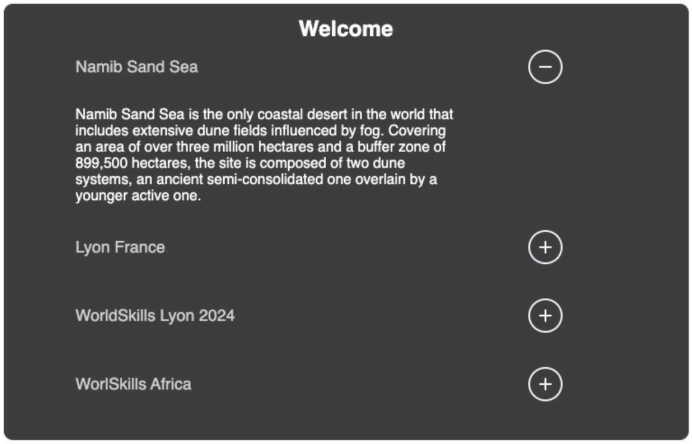
Develop an HTML page as shown in the image below and a JavaScript file that adds functionality to hide and show the password. You can edit the CSS file provided if you want to.



B20: Accordion (Level 3)

You should develop an accordion page using html and JavaScript to add functionality to display the corresponding content as shown in the provided images below when a section header (e.g Namib Sand Sea) is clicked, it should display the corresponding content. You should use the provided CSS file.





© WorldSkills International

B21: Layer Paint Board (Level 3)

You need to create a paint board that displays all layers and a layer list where each layer can be viewed.

You should be able to change the order of layers, add, delete, and select layers, and the drawings on each layer should also be displayed.

When you select a layer and a color, and draw on the paint board, the drawing is applied to the selected layer.

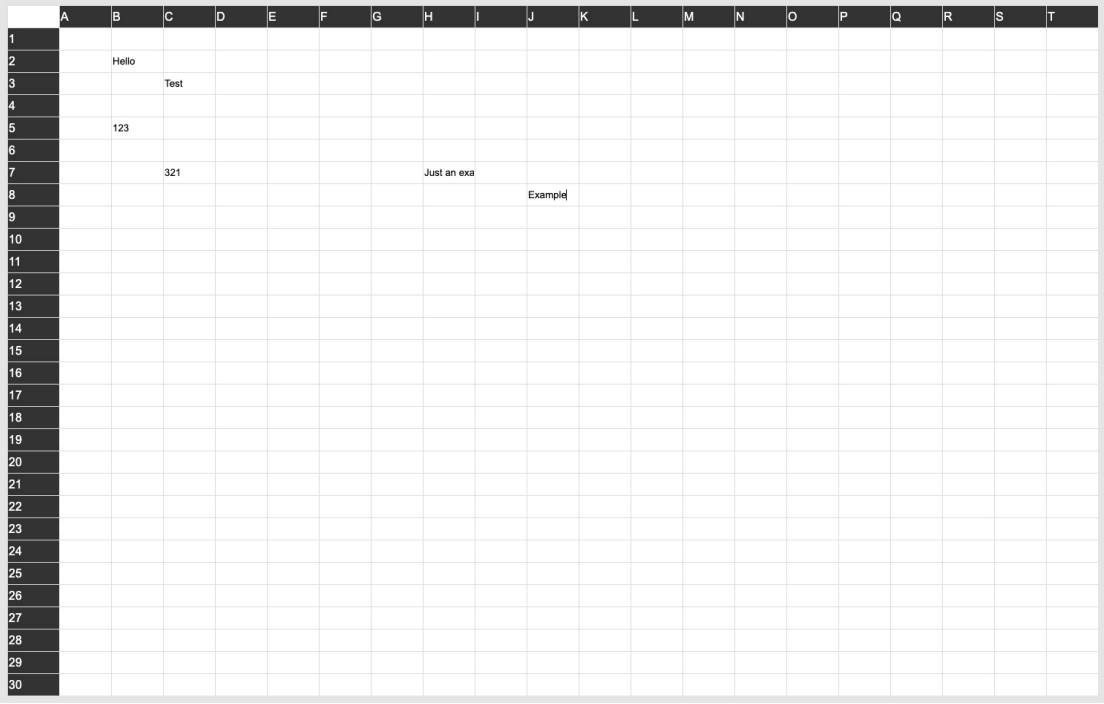
Any changes should be immediately reflected in the layer.

Requirement Notes

- There is a paint board showing all layers, a layer list where you can see each layer, and a layer color picker.
- Can change the order of layers, add layers, delete layers, and select layers.
- When you select a layer and choose a color to draw on the paint board, the drawing is applied to the selected layer.
- The drawing on each layer is displayed.
- Any changes are immediately reflected in the layer.

B22: Excel Table (Level 1)

Please create a prototype Excel table-like web page. User can enter data into each cell.



An Excel table is displayed where you can enter data into cells.

You can see 600 cells at once, with 20 columns and 30 rows.

You can view all cells through scrolling and enter data into the cells.

Columns are labeled with alphabets from A to XFD.

Rows are labeled with numbers up to 1,048,576.

All operations, including data entry and scrolling, should work immediately without delay.

No need for any data saving functionalities.

Requirement Notes

- View and enter data for 600 cells arranged in 20 columns and 30 rows on one screen.
- Scroll to check values up to column XFD and row 1,048,576.
- Data in entered cells remains visible even after scrolling off the screen.
- All operations, including data entry and scrolling, are executed immediately without delay (no pause).

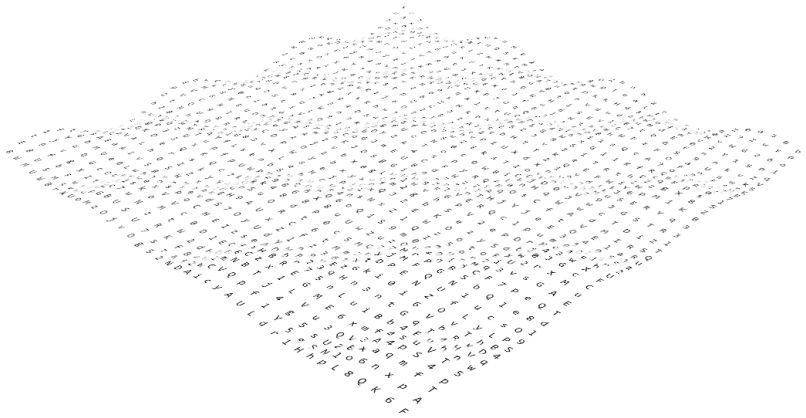
B23: 3D Wave Words (Level 1)

Please create a wave animation with random words consisting of uppercase and lowercase English letters and numbers in a 40x40 grid.

The wave animation should make each random word move up and down in a 3D wave pattern along the Z-axis over time, repeating infinitely.

You can only write code in the provided script.js file.

Please refer to demo.mp4.



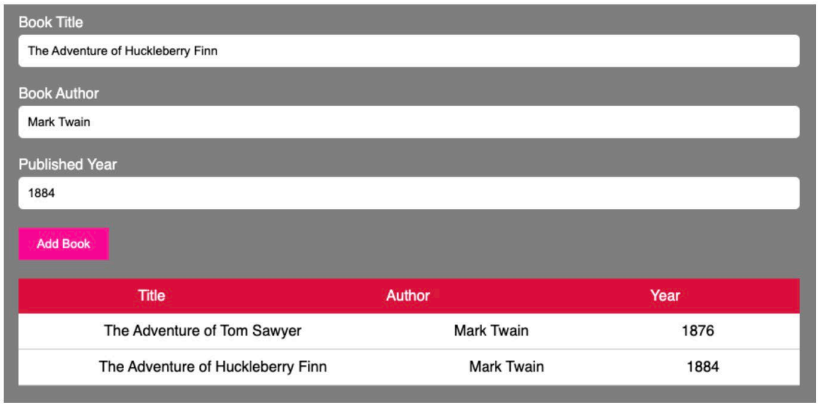
Requirement Notes

- There are 40x40 = 1600 random words consisting of uppercase and lowercase English letters and numbers.
- Each random word moves up and down in a 3D wave pattern along the Z-axis over time, repeating infinitely.
- The functionality was completed by writing code only in the provided script.js file.

B24: Book list (Level 2)

Without touching CSS, develop an HTML page as shown in the image below, and a JavaScript file that adds the functionality to add the book to the list as shown in the images below.





B25: Page transition (Level 3)

You need to implement a transition effect between the given two pages.

Refer to the provided video to implement the animation, and the URL should also change when the page is transitioned.

The provided html cannot be modified, but there is an editable area for css.

Please refer to demo.mp4

Requirement Notes

- Use the provided file
- When you click on the page transition link, a page transition animation similar to the provided video is implemented.
- The url changes with the page transition animation and the page does not change even if the page is refreshed.

© WorldSkills International

B26: Digital Display Board (Level 2)

There is a form to enter text and a place where the entered text will slide in a dot format

The entered text slides from right to left, and when it completely slides out of view, it reappears on the right and slides again.

The text appears in a perfect dot shape, and the shape of the entered characters is displayed in a single color. (See the example below)

Example) When "TEST🎶" is entered:



Requirement Notes

- There is a form to input text and a slide area.
- When you enter text, the input text is displayed in dot form in the slide area (including emojis).
- The dots are a single color and have a perfect dot shape.
- The dot-form text slides from right to left, and when it completely disappears from view, it slides again from right to left (infinite loop).

B27: Sliding Puzzle (Level 1)

Create a 4x4 sliding puzzle using the provided `image.jpg`.

The initial state should have one empty space at the bottom right, and you can click on the pieces around the empty space to move them into the empty space.

When the pieces move, they should move with a smooth animation.

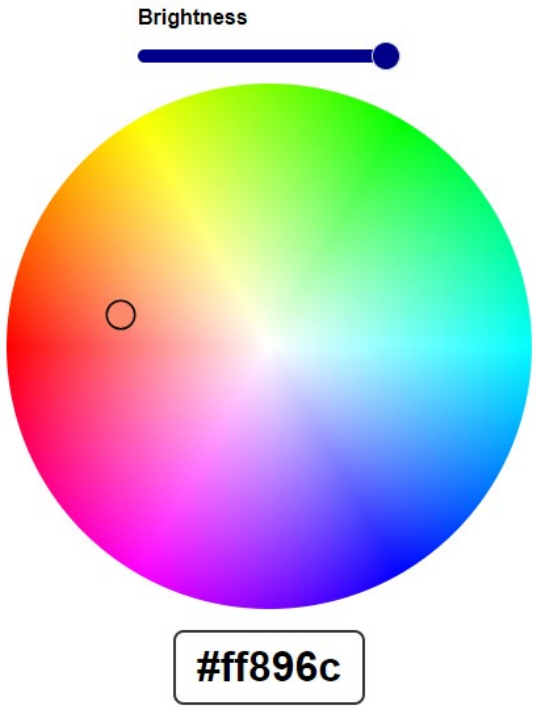
Requirement Notes

- In the initial state, a 4x4 puzzle using the given image is displayed (the bottom right piece is missing).
- When you click on a piece adjacent to the empty spot, the clicked piece moves to the empty spot.
- The movement occurs with a sliding animation.
- If there is no empty spot adjacent to the clicked piece, nothing happens.

B28: Color wheel (Level 2)

When a color is selected, the selected color is displayed as a hex value, and you can see where the selected part is on the color wheel.

Adjusting the brightness will change the color of the color wheel and the selected color value accordingly. Brightness can be adjusted using a slider.



Requirement Notes

- A circular color wheel, a brightness adjuster, and a hexadecimal result value are displayed.
- Clicking on the circular color wheel shows the currently selected color with a marker.
- You can adjust the brightness using the brightness adjuster, which is reflected in the color wheel and the hexadecimal result value.
- When you select a color on the color wheel and adjust the brightness, the hexadecimal result value changes instantly to the correct value.

B29: Firefly (Level 2)

There are 125 cubes arranged in a 5x5x5 3D grid.

Each cube lights up in sequence: first the center cube, then the surrounding 26 cubes, and finally the outermost 98 cubes.

You can rotate a set of cubes by dragging with the mouse to view them from different angles, and you can zoom in and out using the scroll wheel.

Please refer to demo.mp4

© WorldSkills International

Requirement Notes

- 125 cubes are arranged in a cubic form within a 3D grid.
- The cubes light up sequentially from the innermost cube to the outermost cube.
- You can rotate the cube set to view it from different angles by dragging the mouse.
- You can zoom in and out towards the center using the scroll wheel.

B30: Upload Progressbar (Level 1)

After selecting a file, pressing the upload button sends a request to upload.php with that file, displaying the upload progress via a progress bar.

The upload progress should be affected by the network conditions.

Requirement Notes

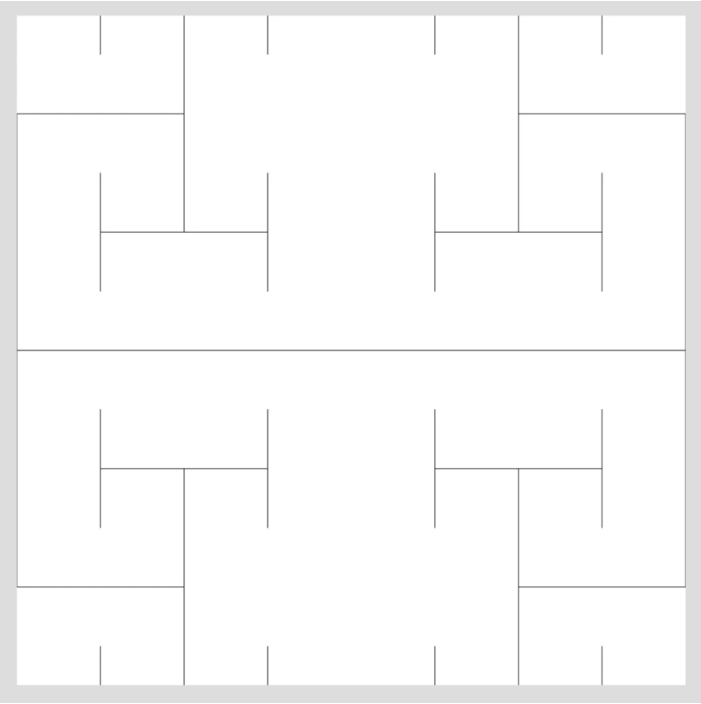
- After selecting a file, clicking the upload button sends the file to upload.php.
- When the upload is complete, the progress bar is filled.
- The upload progress is displayed as a progress bar, reflecting the actual network conditions.

Notes:

- Browser throttling can be used to verify.

B31: Fractal line (Level 1)

After entering the number of iterations and pressing the button, you should draw the Fractal structure by repeating it the specified number of times.



Please refer to demo.mp4

Requirement Notes

- There is a canvas area, a form to input numbers, and a button.
- After entering the number of iterations and clicking the button, a fractal image is drawn.
- The drawn image has the same fractal structure as shown in the video.

B32: Pointer tail (Level 2)

Create a new pointer to replace the existing mouse cursor.

The new pointer is a red circle, and when it moves, it has a tail-like line that follows it.

As the mouse moves and the line appears, the pointer gradually gets smaller, and when the line disappears, it returns to its original size.

Please refer to demo.mp4

Requirement Notes

- The default mouse cursor is hidden and replaced with a new red cursor.
- The cursor's circle becomes smaller when the mouse is moved.
- A tail follows the previous path of the cursor when the mouse is moved.
- The faster the mouse is moved, the longer the tail becomes.

B33: Shared Animation Video (Level 3)

Eight videos are provided. Each video plays automatically on an infinite loop at a 16/9 ratio. When you click on a video, it redirects to the video page.

When transitioning between pages, the following conditions must be met:

- Page reload should not occur. (SPA)
- The currently playing video should not be reset or flicker.
- Videos other than the selected one should fade in/out naturally.
- The video page should display the video in full screen. When the video is clicked again, it should transition back to the initial list page.
- Scrolling on the list page or refreshing on the video page should work the same.

An .htaccess file is provided to enable page navigation without using server-side code. Use this file to ensure the video continues playing even when refreshing on the video page.

Please refer to demo.mp4

Requirement Notes

- On the list page, the 8 videos provided are played in proportion to 16/9.
- When you click on a video on the list page, the area around the video becomes blurred and the video goes full screen with the same transition effect as demo.mp4.
- When the video switches to full screen, the video does not reset or blink.
- If you refresh the video in full screen mode, the video will play in full screen mode.
- When you click on a video in full screen mode, it moves to the list page and the all videos plays again.

B34: Asynchronous calculation (Level 1)

When you input a desired number n and start the calculation, the sum of all numbers from 1 to n appears in the `result`.

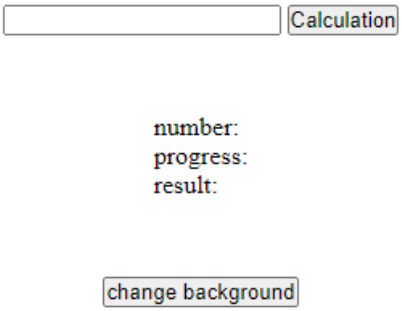
The number entered by the user is displayed in `number`, and the calculation percentage is shown in `progress`.

While calculating, if you press the `Calculation` button, the message `Already calculating` is displayed and does not affect the ongoing calculation.

The calculation should not affect the current browser UI operation. To test this, press the `change background` button to change the background color.

The calculation must use the following logic (you can modify the function but not the calculation logic):

```
function sumNumber(n) {
    let result = 0;
    for (let i = 1; i <= n; i++) {
        result += i;
    }
    return result;
}
```



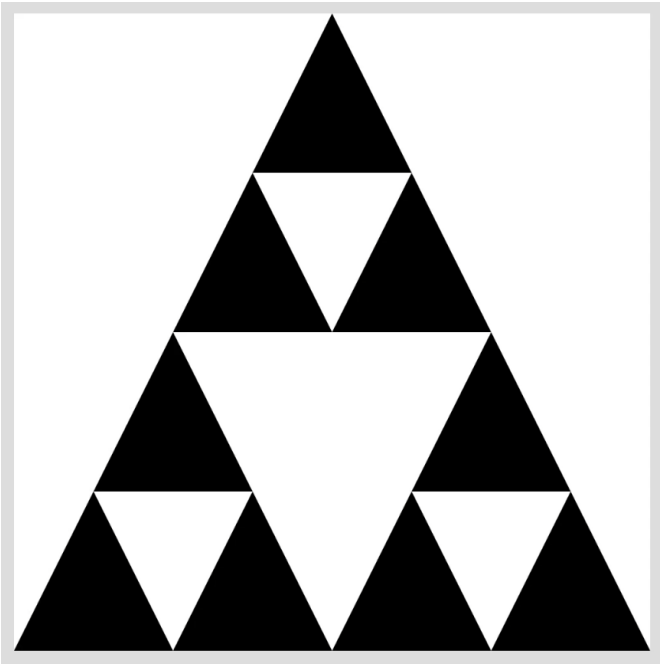
Requirement Notes

- Used the provided HTML and CSS files.
- When you click Calculation, the result is calculated accurately.
- While calculating, the progress increases in proportion to the calculation time.
- While calculating, when you press the change background button, the background color changes randomly.

© WorldSkills International

B35: Fractal Triangle (Level 1)

After entering the number of iterations and pressing the button, you should draw the Fractal structure by repeating it the specified number of times.



(Refer to demo.mp4)

Requirement Notes

- There is a canvas area, a form to input numbers, and a button.
- After entering the number of iterations and clicking the button, a fractal image is drawn.
- The drawn image has the same fractal structure as shown in the video (equilateral triangle structure).

B36: Build Option (Level 2)

There is a simple application made with vanilla JavaScript. The application consists of two connected HTML files.

You need to modify vite.config.js to build the application according to the following requirements.

You can only modify vite.config.js

node_modules are provided in a compressed file. Be sure to refer to README.md

Build Requirements

- Before building, the source files are located at `/xx_module_a/build_option/source`.
- The build output should work correctly at `/xx_module_a/build_option/app/page1.html` without any additional modifications or commands.
- Both CSS and JS should be correctly imported and built.

Requirement Notes

- The source files are in the correct location before building.
- The build output is in the correct location and can be checked through the browser (page1.html)
- Both JS and CSS files are imported correctly, allowing navigation between page1.html and page2.html.

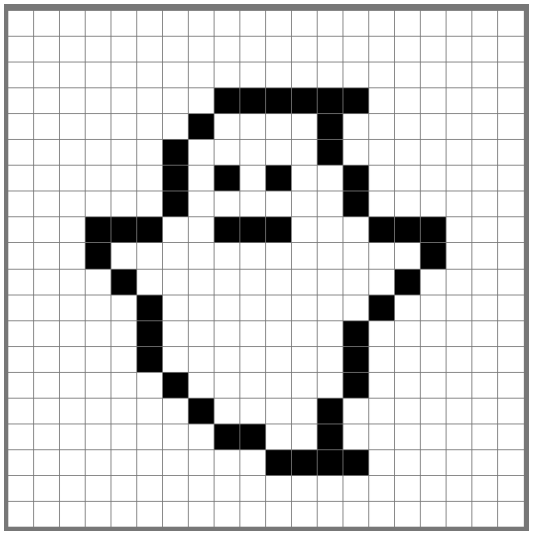
B37: Pixel Art (Level 3)

There is a pixel art drawing board of 20 x 20 size

You can draw and erase with the desired color on the drawing board.

When you press the `Copy` button, the pixel art drawn by the user is converted to the css `box-shadow` property and copied to the clipboard.

Below is an example of pixel art being converted to the css `box-shadow` property and copied to the clipboard. (The class name and width and height must all be copied together.)



```
.pixel-to-css {
    box-shadow: 90px 40px 0 0 #000000, 100px 40px 0 0 #000000, 110px 40px 0 0 #000000, 120px 40px 0 0 #000000, 130px 40px 0 0 #000000, 140px 40px 0 0 #000000, 80px 50px 0 0 #000000, 130px 50px 0 0 #000000, 70px 60px 0 0 #000000, 130px 60px 0 0 #000000, 70px 70px 0 0 #000000, 90px 70px 0 0 #000000, 110px 70px 0 0 #000000, 140px 70px 0 0 #000000, 70px 80px 0 0 #000000, 140px 80px 0 0 #000000, 40px 90px 0 0 #000000, 50px 90px 0 0 #000000, 60px 90px 0 0 #000000, 90px 90px 0 0 #000000, 100px 90px 0 0 #000000, 110px 90px 0 0 #000000, 150px 90px 0 0 #000000, 160px 90px 0 0 #000000, 170px 90px 0 0 #000000, 40px 100px 0 0 #000000, 170px 100px 0 0 #000000, 50px 110px 0 0 #000000, 160px 110px 0 0 #000000, 60px 120px 0 0 #000000, 150px 120px 0 0 #000000, 60px 130px 0 0 #000000, 140px 130px 0 0 #000000, 60px 140px 0 0 #000000, 140px 140px 0 0 #000000, 70px 150px 0 0 #000000, 140px 150px 0 0 #000000, 80px 160px 0 0 #000000, 130px 160px 0 0 #000000, 90px 170px 0 0 #000000, 100px 170px 0 0 #000000, 130px 170px 0 0 #000000, 110px 180px 0 0 #000000, 120px 180px 0 0 #000000, 130px 180px 0 0 #000000, 140px 180px 0 0 #000000;
    width: 10px;
    height: 10px;
}
```



Please refer to demo.mp4

Requirement Notes

- There is a pixel art board size 20 x 20, and each cell must be visible and distinct.
- Each pixel can be changed to any color you want or erased.
- There is a Copy button, and clicking it should copy the box-shadow code of the same image as the Painter image, which can be used in CSS, to the clipboard.
- The copied code has the same format as the specified code. (Spacing doesn't matter)

B38: Masonry Layout (Level 2)

Please implement a gallery layout with 4 columns.

Images are filled into the columns that are the least filled, in order from the leftmost column.

There are 100 images, and 10 images are loaded at a time (in order from 1).

When the last image is reached, another 10 images are loaded. If the last image is visible even after loading, it will load again

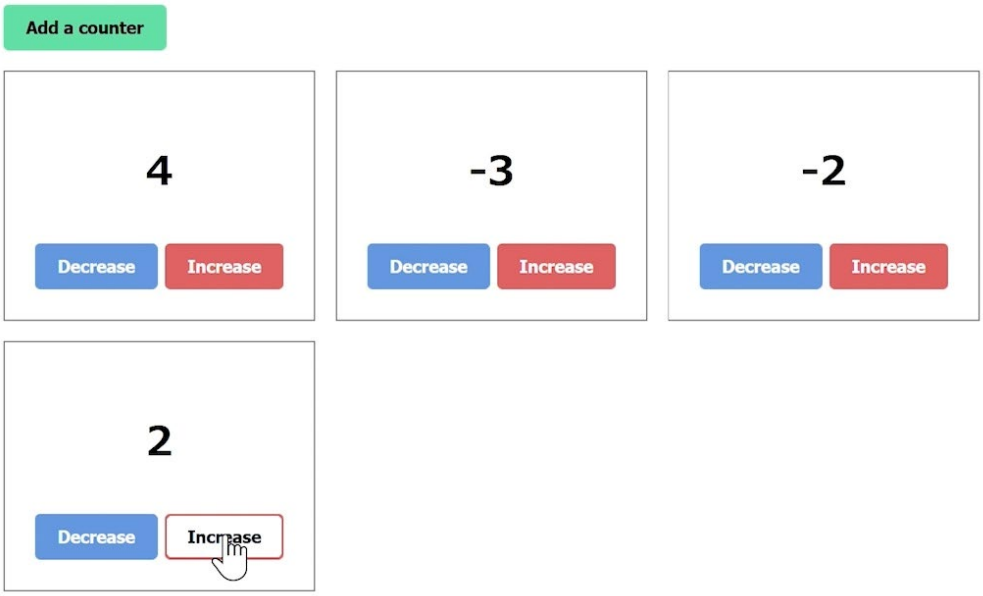
Please refer to demo.mp4

Requirement Notes

- A gallery layout with 4 columns is displayed.
- 100 images are arranged in order (images are placed in the column with the smallest height first, and if columns have the same height, they are filled from the leftmost column).
- When the last image is visible, the next ten images are loaded.
- If the last image is still visible after ten images are loaded, the next ten images are loaded immediately.

B39: Multiple Counters (Level 1)

Using the provided video as a reference, complete the function using JavaScript.



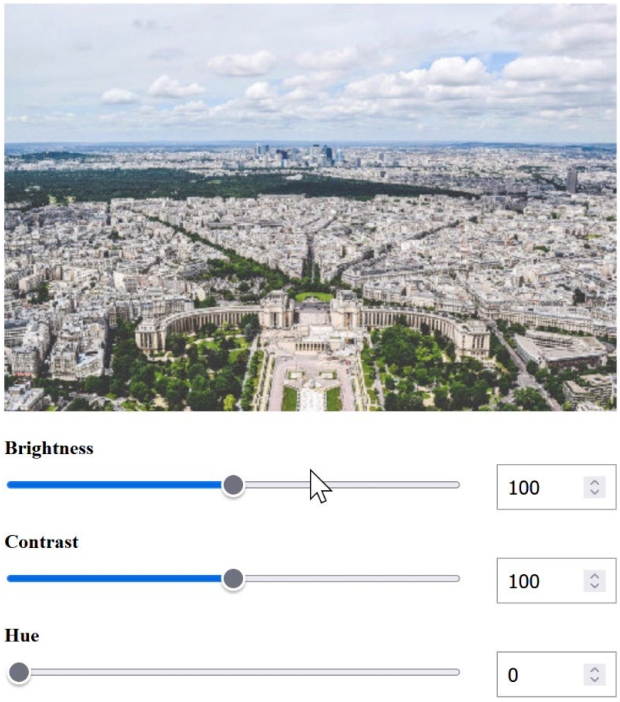
Requirement Notes

- Can add a counter
- Can add multiple counters
- Each counter can independently increase and decrease
- UI is adjusted with user experience in mind

B40: Image Editor (Level 2)

Using the provided video as a reference, please create an image editor that can change brightness, contrast, and hue using JavaScript.

© WorldSkills International



Requirement Notes

- Brightness works correctly
- Contrast works correctly
- Hue works correctly
- Can be adjusted using numeric input

B41: Change Date Format (Level 1)

Using the provided video as a reference, please complete the function using JavaScript.



Requirement Notes

- Select changes days based on the month
- Results update automatically when selection changes
- Converts and displays results ('-' separated, adding leading zeros if single digit)
- UI is adjusted with user experience in mind

Tasks for Back-end Development

C1: Folder Zip (Level 1)

Please implement a simple web page for compressing folders. Users can select a folder (not empty) and click the "Compress" button to compress it into a zip file (The file name is the same as the uploaded folder name) and automatically download it. If the uploaded folder contains empty subfolders, those subfolders will not be included in the compressed archive. The folder for testing will be provided as media files

C2: WSC News(Level 2)

Given the following HTML:

index.html

```
<div class="container">
    <ul class="news-list">
        <li>
            <img src="./imgs/worldskills-museum-opening-thumb.jpg" alt="">
            <div class="news-item-text">
                <h3><a href="news.html">{ Title }</a></h3>
                <p>
                    { Excerpt }
                </p>
                <p class="news-date">{ Date }</p>
            </div>
        </li>
    </ul>
    <ul class="pages">
        <li>
            <a href="#" class="previous">Previous</a>
        </li>
        <li><a href="#" >1</a></li>
        <li><a href="#" >2</a></li>
        <li><a href="#"  aria-current="page">3</a></li>
        <li><a href="#" >4</a></li>
        <li><a href="#" >5</a></li>
        <li>
            <a href="#" class="previous">Next</a>
        </li>
    </ul>
</div>
```

news.html

```
<div class="container news-page">
    <h1>{ Title }</h1>
    <p>
        { Description }
    </p>
    <figure>
        <img src="./imgs/WorldSkillsGlobalResearchCouncil.jpg" alt="">
    </figure>
    <p class="news-date">{ Date }</p>
</div>
```

And given the following CSS (style.css):

```css
* {
    margin: 0;
    padding: 0;
    font-family: Arial;
}
a {
    text-decoration: none;
}
ul { list-style: none; }
.container {
    width: 1200px;
    margin: 0 auto;
}
.news-list {
    min-height: 664px;
}
.news-list li {
    padding: 16px 0;
    display: flex;
    align-items: center;
    gap: 16px;
}
.news-list li img {
    width: 100px;
    height: 100px;
    object-fit: contain;
}
.news-list li a {
    color: deeppink;
}
.news-list li a:hover {
    text-decoration: underline;
}
.news-list li:not(:last-child) {
    border-bottom: 1px solid rgba(0,0,0,0.2);
}
.news-item-text p {
    margin-top: 10px;
}
.pages {
    display: flex;
    align-items: center;
    gap: 0;
}
```

```css
.pages a {
    display: block;
    cursor: pointer;
    border: 1px solid deeppink;
    color: deeppink;
    padding: 8px;
    width: 20px;
    height: 20px;
    line-height: 20px;
    text-align: center;
    transition: .2s;
}
.pages a[aria-current],
.pages a:hover {
    background-color: deeppink;
    color: white;
}
.previous, .next {
    width: 60px!important;
}
.news-date {
    color: #999;
    font-style: italic;
}
.news-page {
    display: flex;
    flex-direction: column;
    gap: 20px;
    padding: 20px 0;
}
.news-page h1 {
    font-size: 40px;
    text-align: center;
    color: deeppink;
}
.news-page img {
    max-width: 500px;
}
.back {
    text-decoration: underline;
}
```

And given the following JSON's schema (data.json):

```
[
    {
        "img": "Image's Name, like: worldskills_img.jpg",
        "title": "Title of the news",
        "excerpt": "Excerpt of the news",
        "date": "Date of the news",
        "id": "The new's ID",
        "description": "Description of the news"
    },
    ...
]
```

Please generate a static website (not a single-page application) using the provided HTML template files, based on the news data provided in data.json. The website should include pagination for the news list, displaying 5 news per page. Each news in the list should only display the news title, excerpt, and date. Clicking on a news item in the list should redirect to the news details page, which displays the news title, description, and date.

The template file for the news list page is index.html, and the template file for the news details page is news.html.

For the pagination functionality on the news list page, please ensure that the link for the current page is highlighted. Additionally, the Previous button should be disabled when on the first page, and the Next button should be disabled when on the last page.

C3: CSV converter (Level 2)

Create a script that takes the path to a <path/to/some-name>.csv-file as query-parameter and returns a <path/to/some-name>.json-file into the same path. The script should convert the content of the csv-input (list of records) into a JSON-output. The first line of the csv contains a list of columns to be considered dynamically for the conversion. Empty values in a record are stored as "". An example for csv and the resulting json are provided.

Requirement Notes

- Output in same directory with same basename as given as input parameter (extension differ)
- Dynamic number of columns respected
- Input with same or similar data as example produces output as expected

C4: String reverser (Level 1) (PHP Version)

Using PHP, create the following task: take a string input (e.g., "worldskills") from a user via an HTML form, reverse the string, convert it to uppercase, and display the result on the same page without reloading.



Requirement Notes

- The form correctly submits the input string using the POST method.
- The form and PHP code handle input validation correctly, ensuring that only non-empty strings are processed
- The input word is converted to upper case
- The inputted word is reversed

C5: Recurring Calendar (Level 3)

The player must create a recurring calendar. The schedules should be managed in a database, with each schedule (including recurring ones) having a single row.

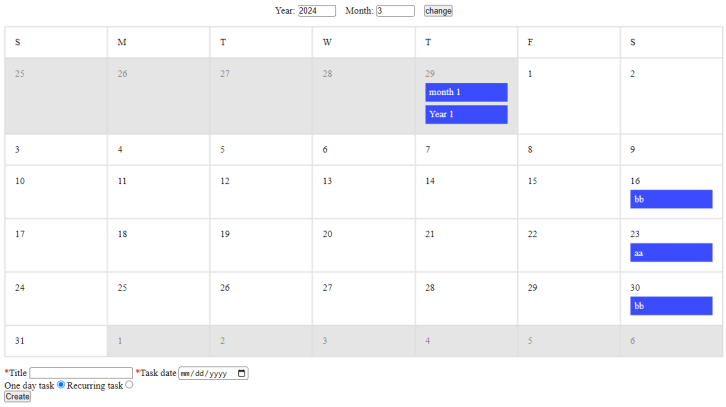
You can select the Title, Task Date, and Is recurring, and only if recurrence is enabled, you can input the Type, Cycle, and End Date. The calendar may include parts of the previous month and the next month.

Type can be one of Day, Week, Month, or Year, and the schedule will repeat every Cycle number of Types.

e.g., if Type is Week and Cycle is 2, the schedule repeats every 2 weeks.

If the date of a recurring schedule exceeds the last day of the month in which it should appear, it will be displayed on the last day of that month.

Users can change the year and month of the displayed calendar.



Requirement Notes

- There is a form to change the date, a calendar for the currently selected month, and a form to add events.
- A single schedule can be added.
- Recurring schedules can be added.
- Single schedules are displayed correctly. 0
- Recurring schedules are displayed correctly (if the day of a recurring schedule is beyond the last day of the month, it is displayed on the last day of the month; only recurrences after the task date are shown on the calendar).
- The date on the calendar can be changed.

C6: SQL Data Import – (Level 1)

You are provided with a CSV file (*console_games.csv*) containing data about console games. Your task is to create a database ***data_import*** and to import all the available data from the CSV file into the database as one table ***console_games***. Provide your solution as one SQL file ***console_games.sql*** and once all the data were imported provide the SQL dump of the database as ***data_import_dump.sql***.

NOTE: If this is handled on a localhost, in order to allow local files upload, it might be required that the my.ini to be edited and this to be added at the end of file and MySQL restarted.

#Enable local files upload

[mysqld]

secure_file_priv=" "

For a live URL server, the absolute path will be required.

Requirement Notes

- A workable solution was provided as SQL file
- The "***sql_import_dump.sql***" was provided with correct naming
- The database name is correct as "***data_import***"
- All the CSV data are imported (10 columns / 15979 last game ID)

© WorldSkills International

C7: Excel Download (Level 3)

Import the provided `database.sql` to set up the database.

When you access the page, the `posts.xlsx` file will be downloaded.

The Excel file should display the id and title of all posts and the number of related comments. (Refer to the format below)

ID	TITLE	COMMENT COUNT
1	post title 1	3
2	post title 2	1

Requirement Notes

- The file posts.xlsx is downloaded.
- The downloaded file can be opened with an Excel program.
- The data stored in the file reflects the data from the database.

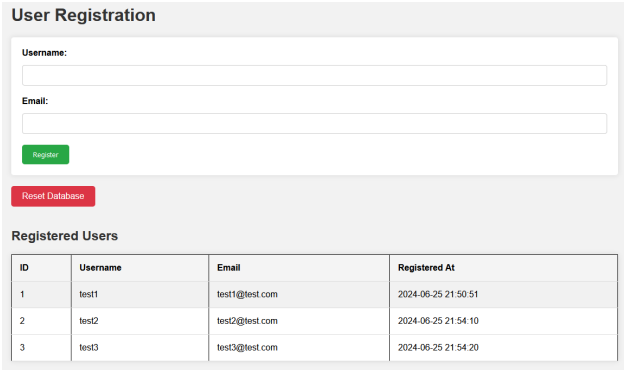
C8: User registration (Level 3)

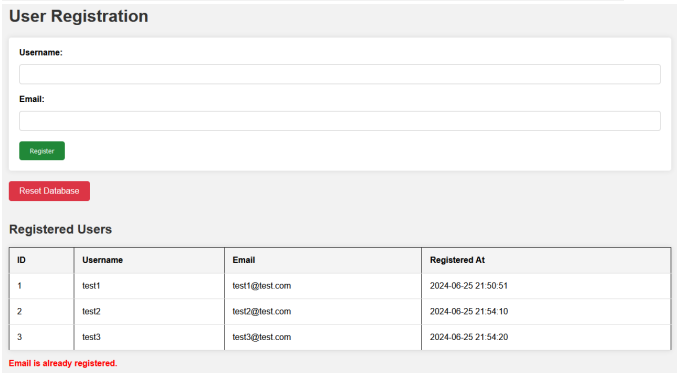
There are two folders provided and you can choose your approach by either using PHP or Node.js + Express.

You have been provided with the layout; SQL file to create the necessary user's table; basic scripts for connection and reset database.

Create a user registration form that allows users to register with their username and email. The system should display a list of registered users and the time of registration (as below). If an email address already exists, the error message "Email is already registered" should be returned.

The required behaviour is presented in the provided media_sample/user_registration.mp4





Requirement Notes

- The `register` button add a new entry to database and is returned in the table (Treat as errir if it is just in the database but not in the frontend)
- The new entry appears in the `registered users` table without page refresh
- The existing email cannot be inserted, and the error message appear
- The database is reseted correctly (Treat as error, for example, when after reset the IDs are not starting from 1)

C9: API CORS (Level 2)

A Front-end application and a Back-end application are provided.

Access the Back-end through the VM Apache server, and use `npm run dev` for the Front-end application, making sure to use a different port than the Back-end.

When accessing the Front-end, 4 methods do not execute properly due to CORS.

Modify only the `cors.php` file in the Back-end application to ensure all requests execute correctly.

Run Front-end Application

You can run the development server by executing the `npm run dev` command.

Request address settings

You must open the front-end's `index.html` and modify the `API_HOST` variable to match your back-end.

(node_modules are provided in a compressed file. Be sure to refer to README.md)

Requirement Notes

- When you open the development server and connect, requests are sent to the back-end application.
- The GET method request works correctly.
- The POST method request works correctly.
- The PUT method request works correctly.
- The DELETE method request works correctly.

C10: Realtime Chat (Level 2)

You need to create a real-time chat app divided into Front-end and Back-end.

Using the provided Front-end and Back-end templates and socket, you should be able to send messages to the currently connected users.

Differentiate between your messages and messages from others, and the input field should be cleared after sending a message.

(node_modules are provided in a compressed file. Be sure to refer to README.md)

Deploy

- Back-end will run the server by executing the `npm run start` command for testing.
- Front-end must operate when accessing the specified URL. `/xx_module_a/realtime_chat`

Requirement Notes

- It was developed separately into back-end and front-end.
- Front-end used the given file.
- Back-end and front-end communicate through socket.io, a given package.
- When you enter a message in the front-end, the message is sent to all currently connected users and can be viewed in real time.
- After sending the message, the input window will be blank.

C11: Image Mosaic (Level 1)

You need to dynamically create a mosaic of the provided image.

The completed image should be made up of square cells, with each cell having the average color value of the corresponding area of the original image.

The size of the cells can be set using the query parameter `cell_size`. If `cell_size` is not provided, it should be set to 50px.

Example) When cell size is 50px:



Requirement Notes

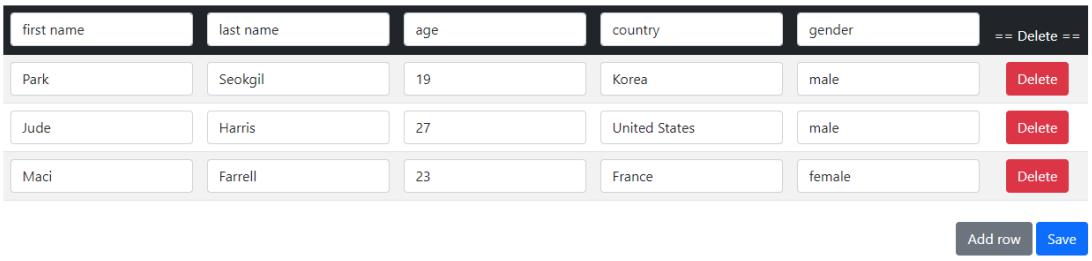
- The mosaic is created according to the size specified by the `cell_size` query parameter.
- If the `cell_size` query parameter is not provided, the mosaic is created with cells of size 50px.
- The mosaic image is the same size as the original image.

C12: Local Table Data Control (Level 1)

Read the provided `table.json` file and display it to the user in table format.

The user can modify the table data (fields, rows) and add or delete rows.

Added or deleted data is saved directly to `table.json`, and modified table data is saved by pressing the save button.



Requirement Notes

- The correct data is displayed to the user based on the provided JSON.
- When the delete or add button is pressed, the changes are immediately reflected in the JSON.
- When the save button is pressed, the modified data is reflected in the JSON (including fields).
- Data that has been deleted, added, or modified is saved correctly according to the existing JSON structure.

C13: Login System Using JSON File (Level 3)

Implement the function using PHP or Node.js based on the following specifications. Do not use a database; instead, save data to a JSON file. JavaScript cannot be used.

- User Registration Function
  - Create an HTML form
- Save User Information
  - Save the username and password sent by the user to the `users.json` file
  - Hash the password before saving
  - Display an error message when attempting to register with empty fields
- Login Function
  - Create an HTML form that allows the user to input username and password and send this information to the server
  - Display an error message when attempting to log in with empty fields
- User Authentication
  - Compare the username and password sent by the user with the saved information
  - If the username and the hashed password match, authenticate the user and display a login success message
  - Display a login failure message if they do not match
- Logout
  - Show the logout button only when login is complete

Requirement Notes

- Can register a user
- Can register multiple users
- Can log in
- Can log out
- Displays a message when submitting empty fields
- UI is adjusted with user experience in mind

Instructions to the Competitor

Each mini Test Project folder should be separated and self-contained.

Please include only exactly the required mini test project. And do not include any not related folders or not related mini Test Projects.

Other

This project will be assessed by using Google Chrome web browser.

Marking Summary

	Sub-Criteria	Marks
1	Mini Test Project General	4
2	Design Implementation	2
3	Front-end Development	2
4	Back-end Development	2

© WorldSkills International